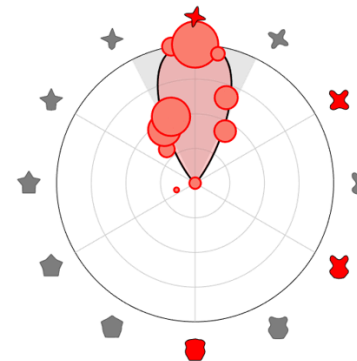
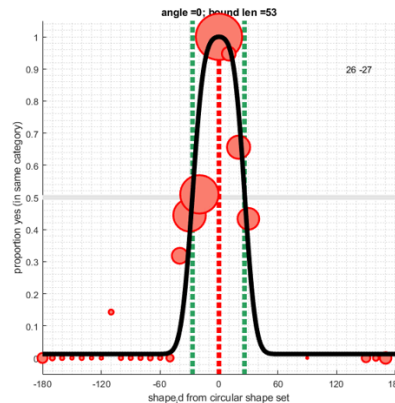
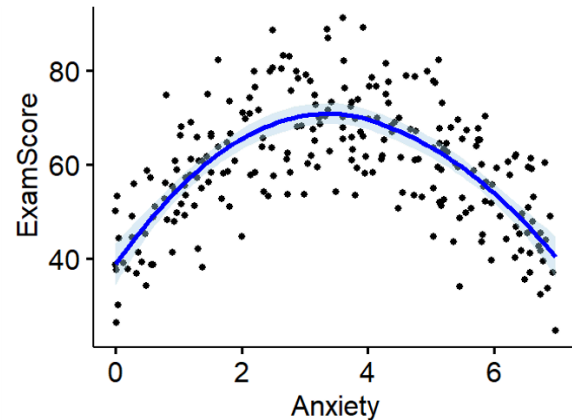
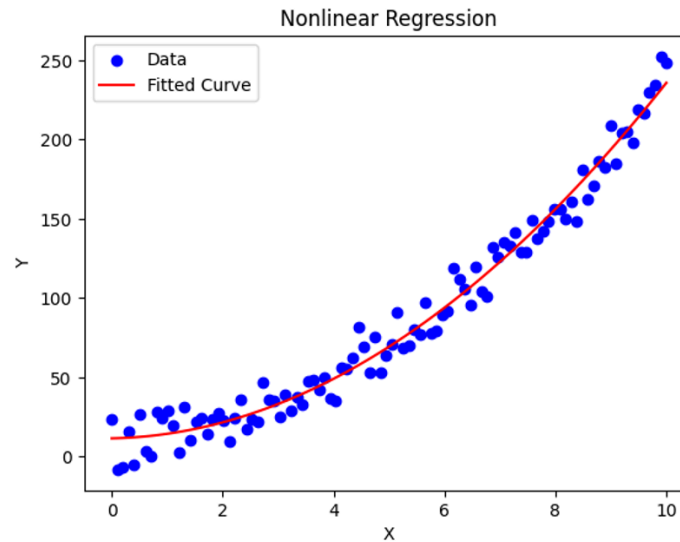
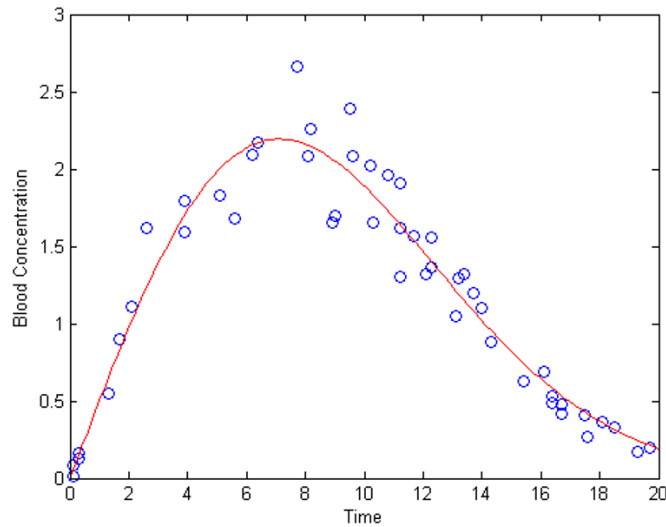


Supervised learning / Parameter Estimation

Where we are

- Today: once you have a model, how do you find the numbers that make it fit your data?

Today: Estimating model parameters from data. Example

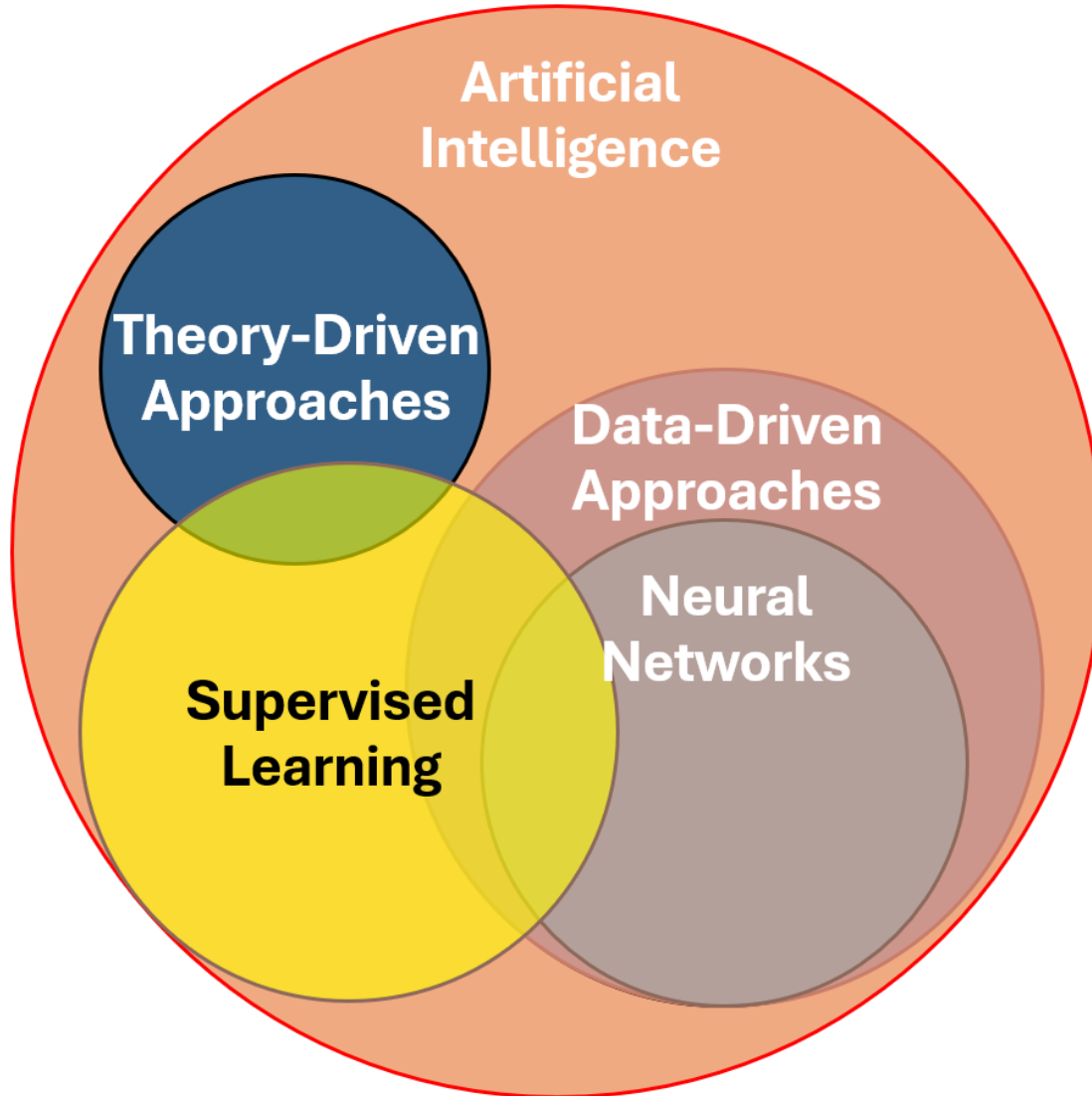


Learning objectives

By the end of today you should be able to:

1. Explain what a parameter is, and why we need to *estimate* it from data
2. Explain what RMSD is, and why data never sits exactly on a fitted curve
3. Use grid search to find best-fitting parameters by hand, and explain why it doesn't scale
4. Explain, at a high level, how `optim()`'s default method (Nelder-Mead simplex) searches — and its limitations, including local minima
5. Fit a model using both RMSD

What is Artificial Intelligence (AI)?



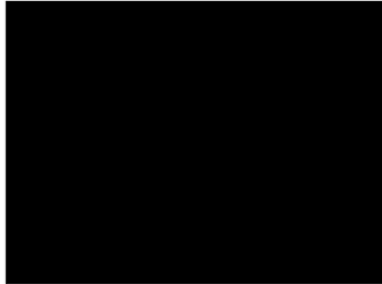
Supervised Learning

Categorical



Bicycle

Continuous



0



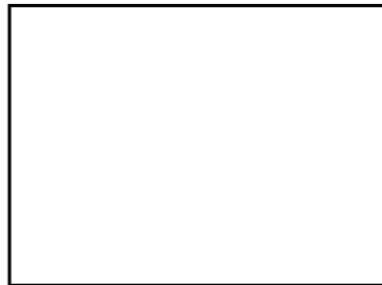
Apple



0.5

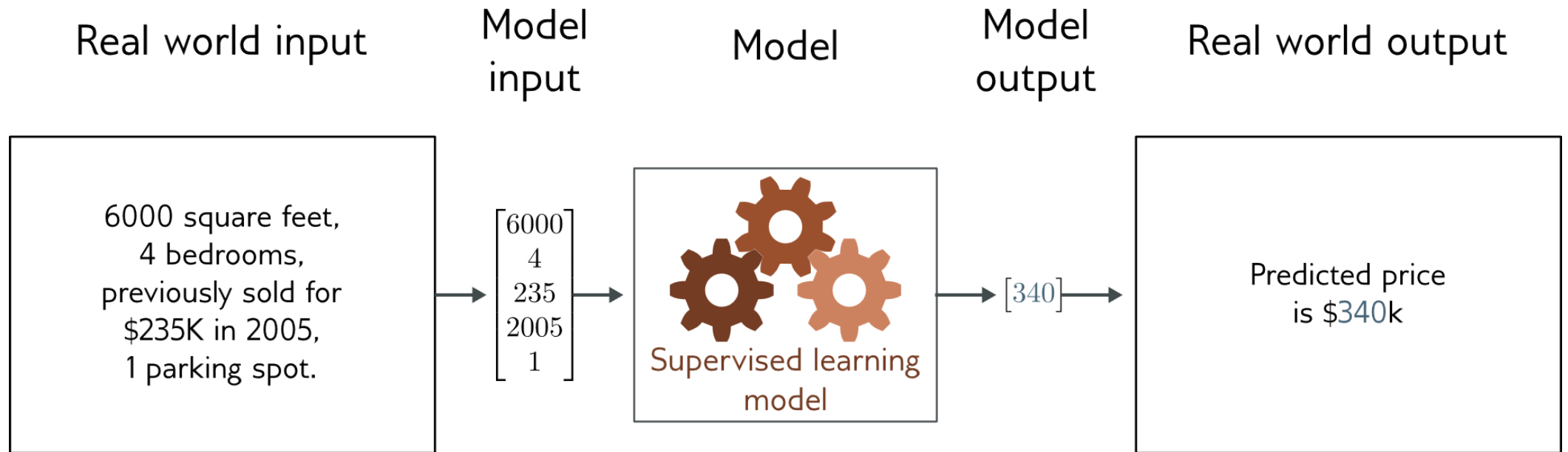


Aardvark



1.0

Regression



- Univariate regression problem (one output, real value)
- Prince, S. J. (2023). Understanding deep learning. MIT press.

Supervised learning overview

- **Supervised learning model** = mapping from one or more inputs to one or more outputs
- Model is a mathematical equation (e.g., $y = \phi_1 x + \phi_0$), or family of equations (e.g., neural networks)
- Model also includes **parameters** (e.g., ϕ_1 and ϕ_0 from above)
- Parameters affect outcome of equation
- **Training** a model = finding parameters that predict outputs “well” from inputs for a **training dataset** of input/output pairs

Supervised learning overview

- Now, let's look at a familiar example
- See [live demos 2.1, 2.2, 2.4](#) from Chapter 2 in Prince, S. J. (2023). Understanding deep learning. MIT press.
 - <https://udlbook.github.io/udlfigures/>

Example: 1D Linear regression model (demo 2.1)

- Model:

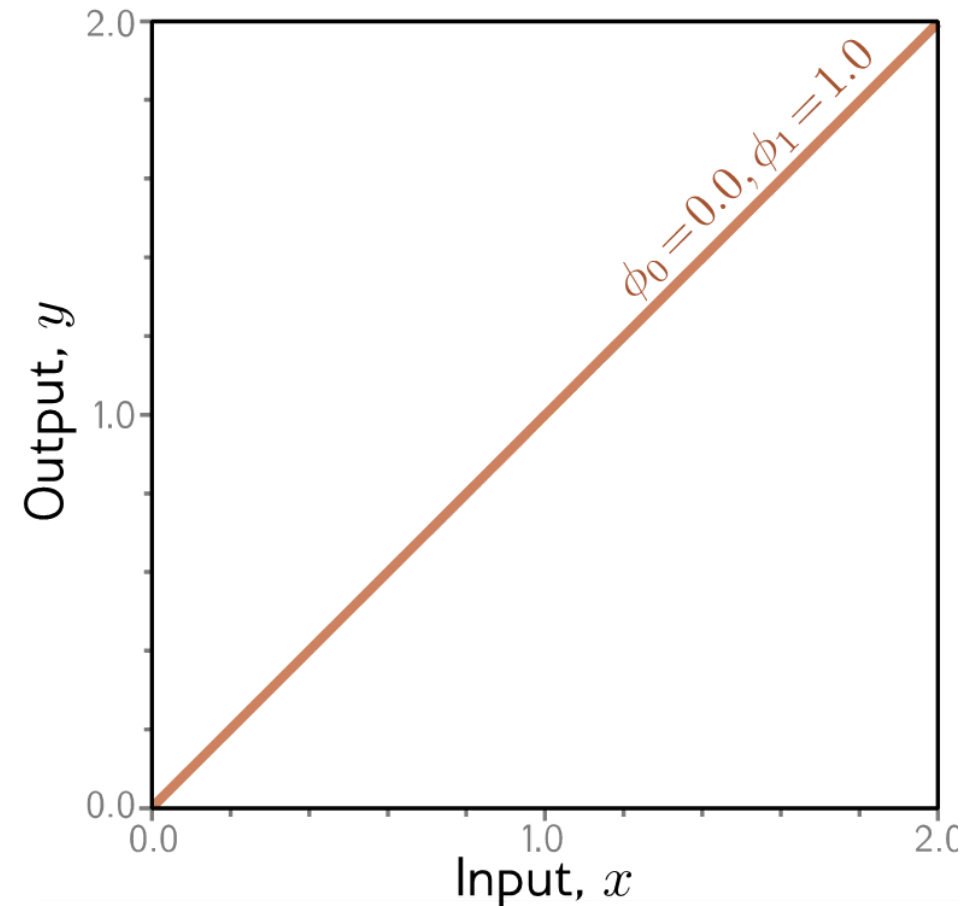
$$\begin{aligned}y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x\end{aligned}$$

- Parameters

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}$$

← y-offset

← slope



Example: 1D Linear regression model (demo 2.1)

- Model:

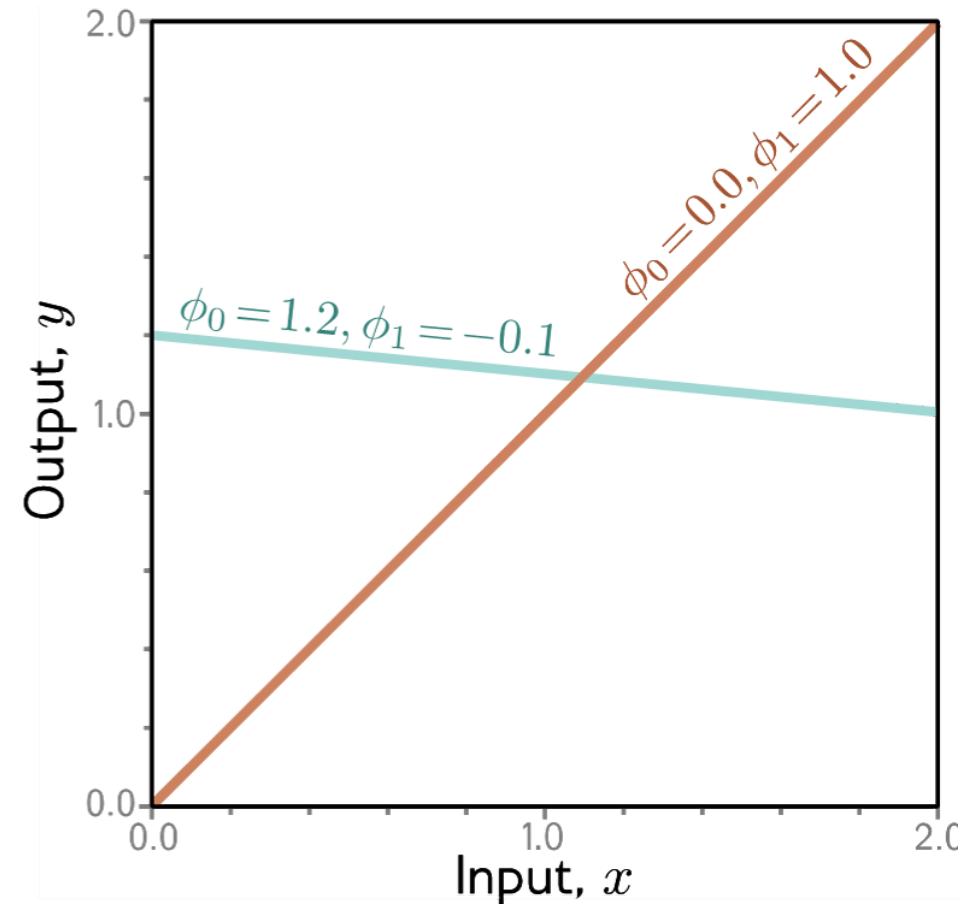
$$\begin{aligned} y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x \end{aligned}$$

- Parameters

$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}$$

← y-offset

← slope



Example: 1D Linear regression model (demo 2.1)

- Model:

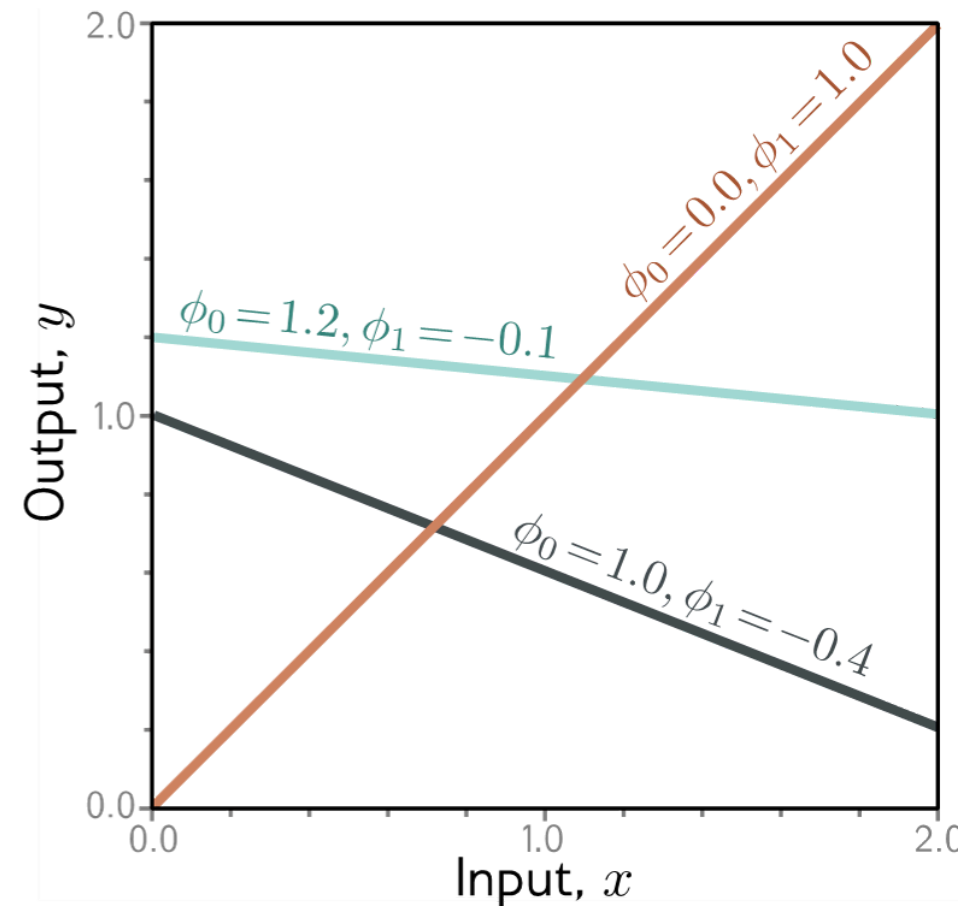
$$\begin{aligned}y &= f[x, \phi] \\ &= \phi_0 + \phi_1 x\end{aligned}$$

- Parameters

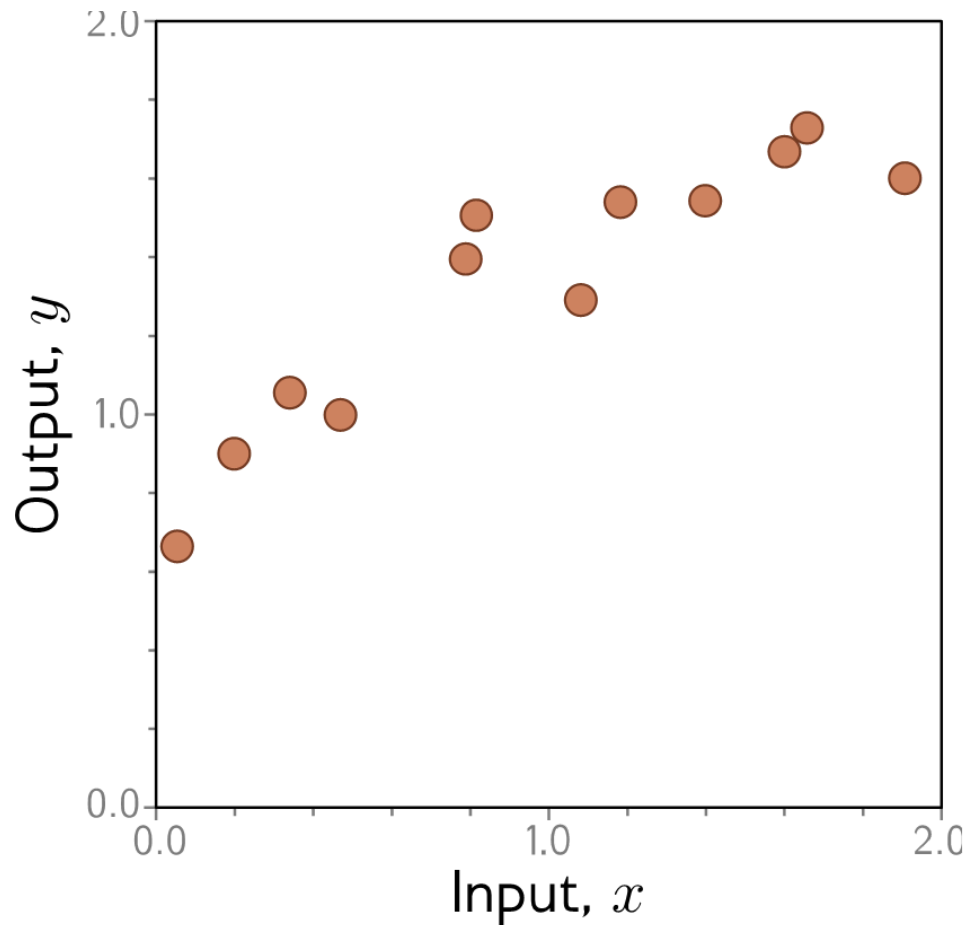
$$\phi = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}$$

← y-offset

← slope



Example: 1D Linear regression model (demo 2.2)

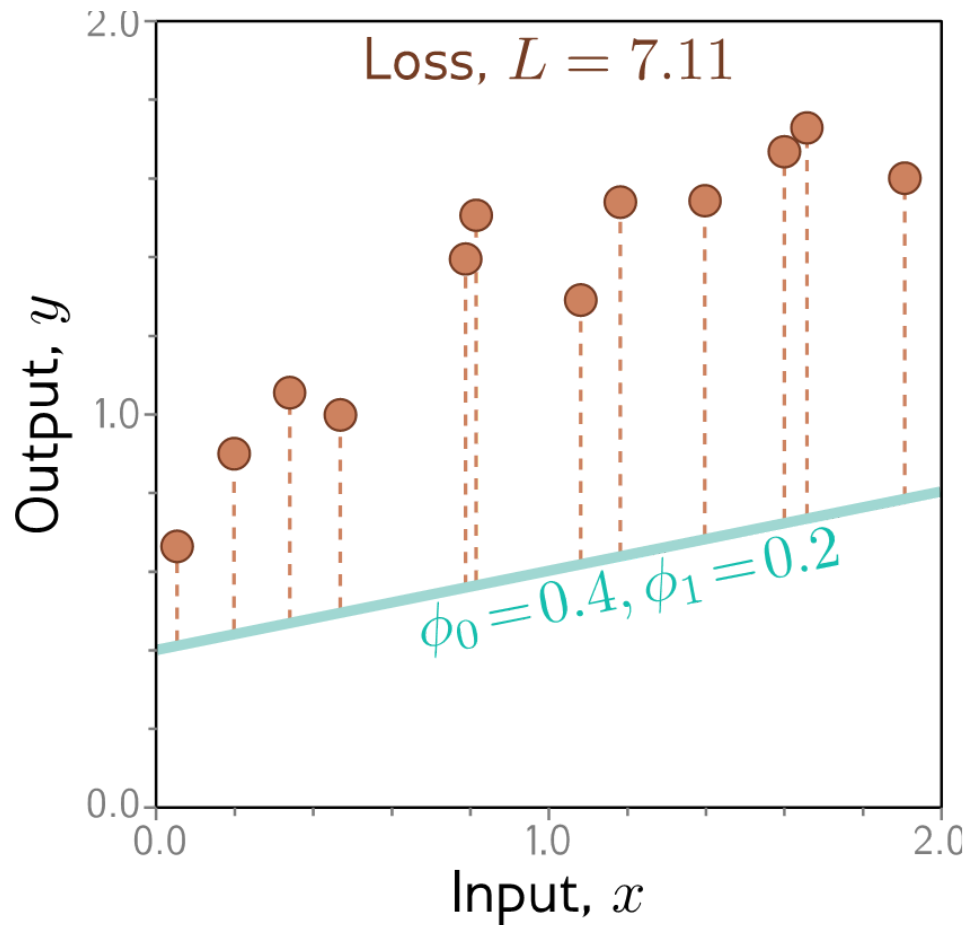


Loss function:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

“Least squares loss
function”

Example: 1D Linear regression model (demo 2.2)

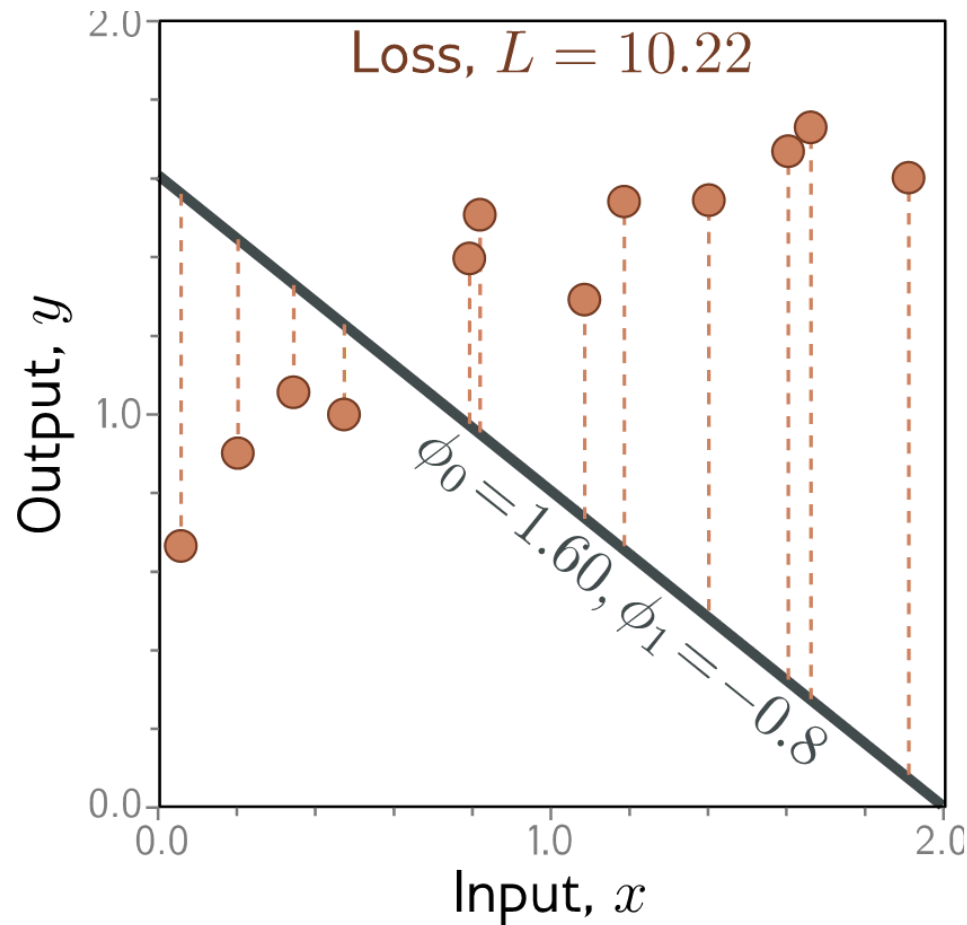


Loss function:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

“Least squares loss
function”

Example: 1D Linear regression model (demo 2.2)

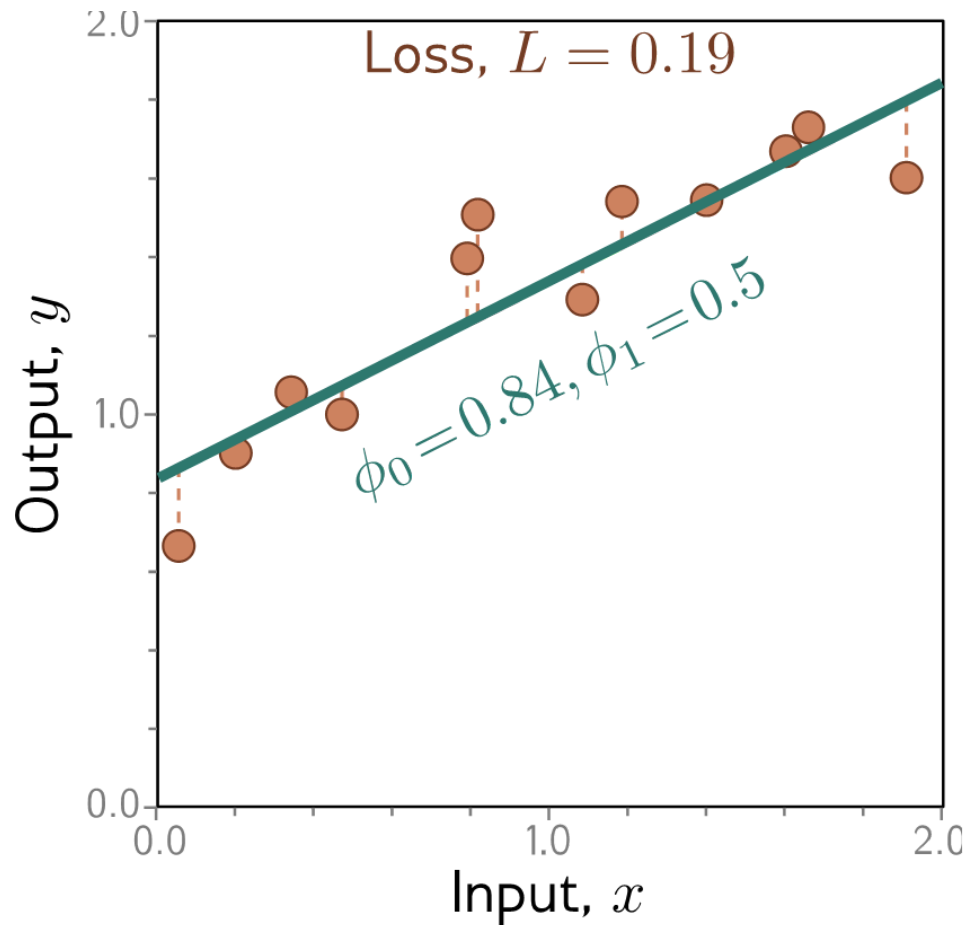


Loss function:

$$L[\phi] = \sum_{i=1}^I (f[x_i, \phi] - y_i)^2$$
$$= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2$$

“Least squares loss
function”

Example: 1D Linear regression model (demo 2.2)

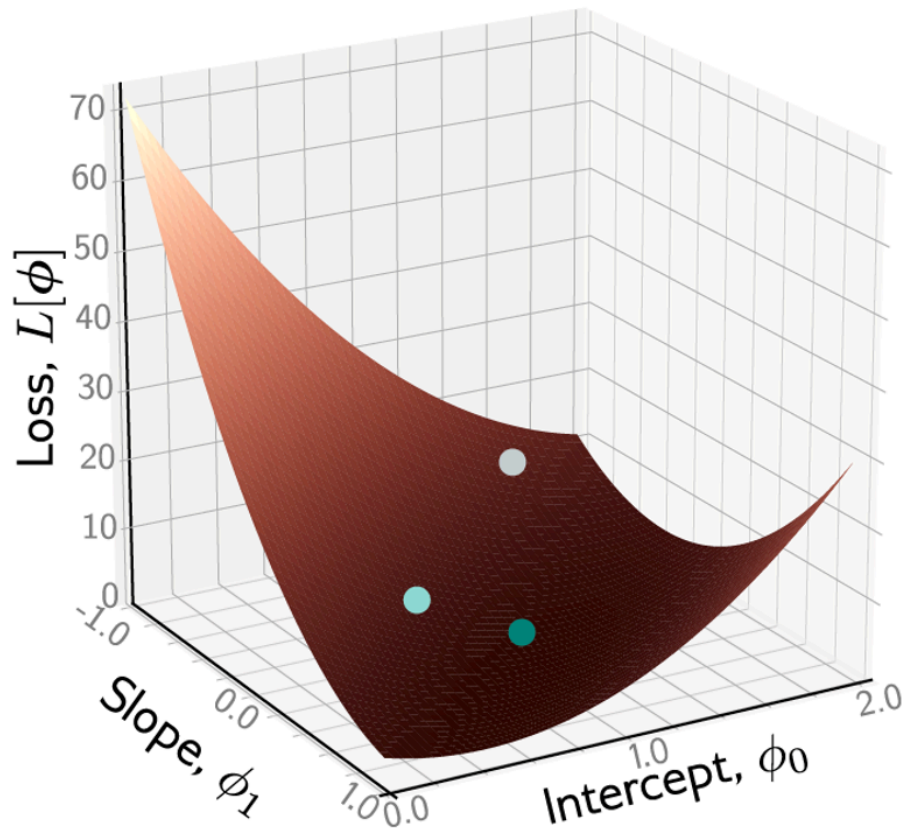


Loss function:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

“Least squares loss
function”

Example: 1D Linear regression model (demo 2.4)



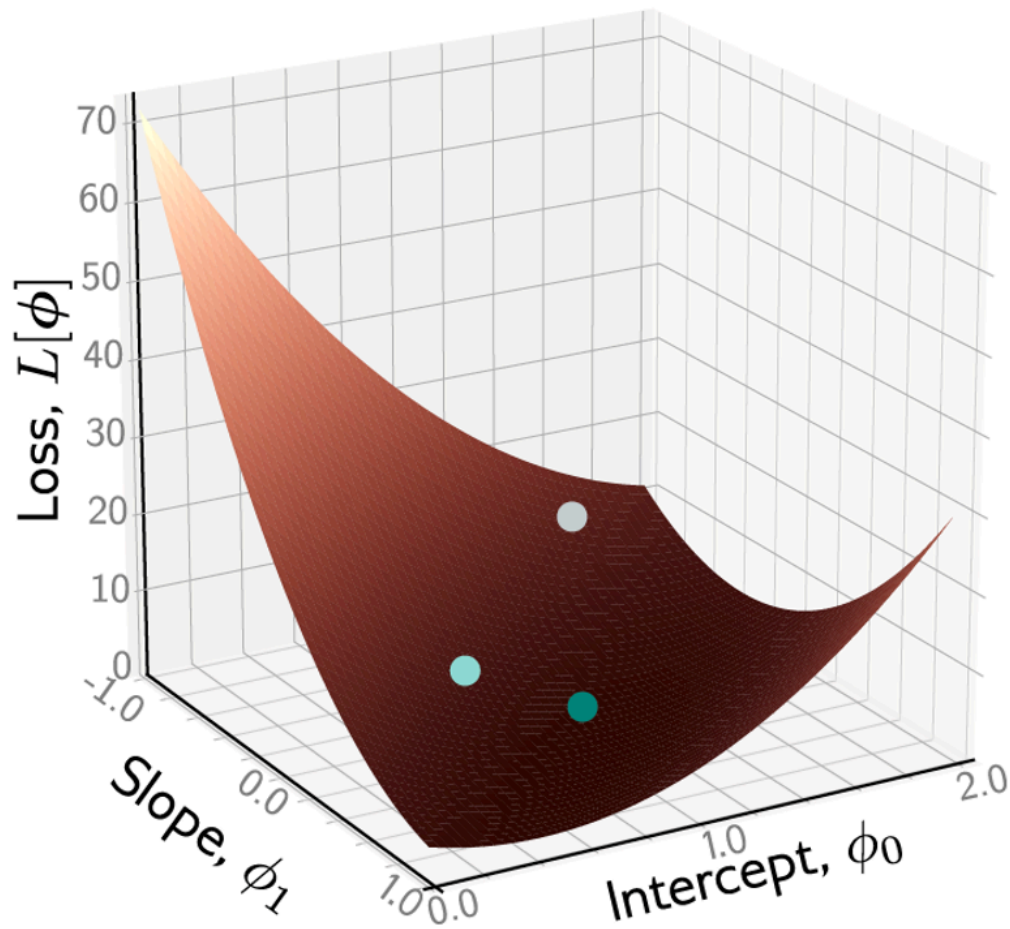
Loss function:

$$\begin{aligned} L[\phi] &= \sum_{i=1}^I (f[x_i, \phi] - y_i)^2 \\ &= \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2 \end{aligned}$$

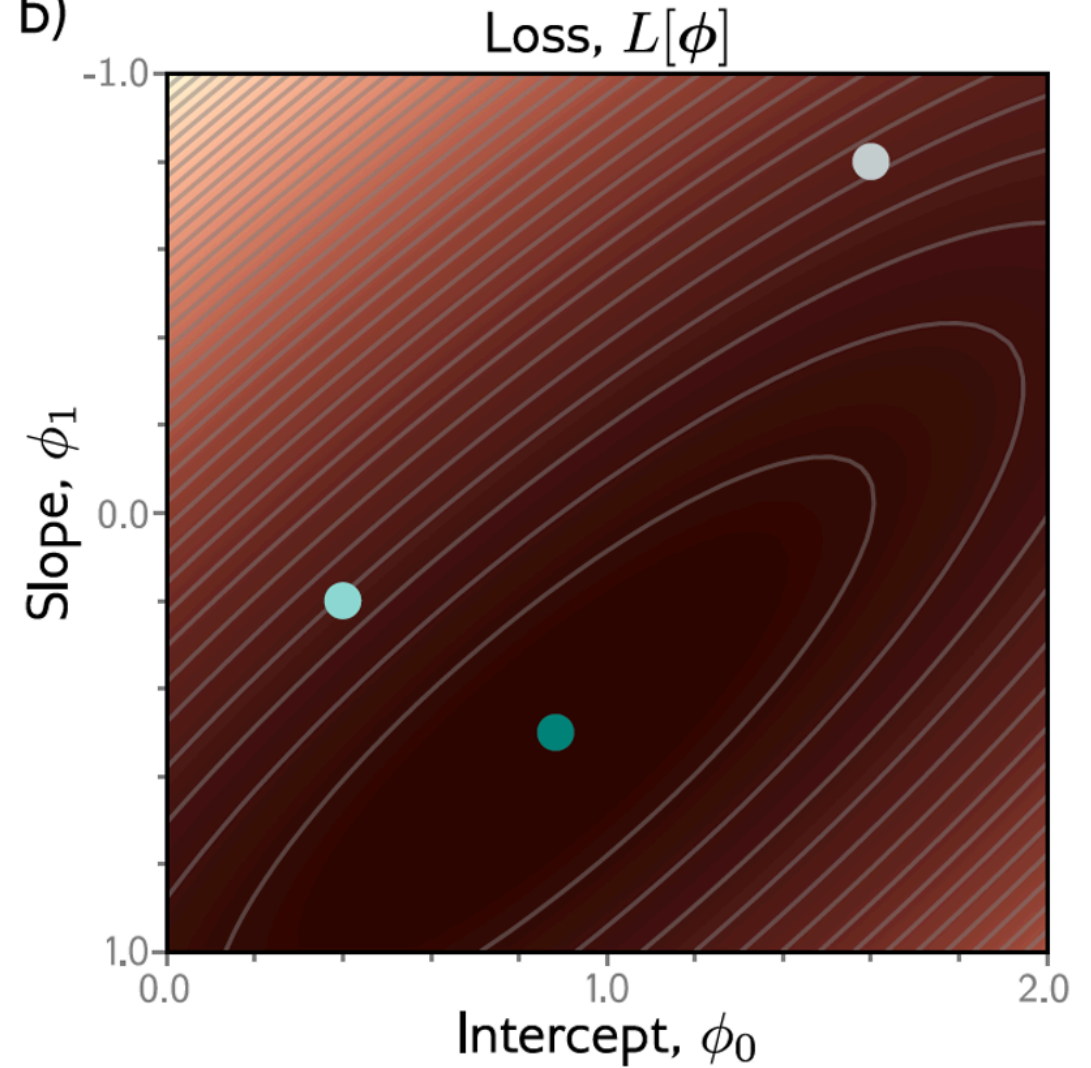
“Least squares loss
function”

Example: 1D Linear regression model

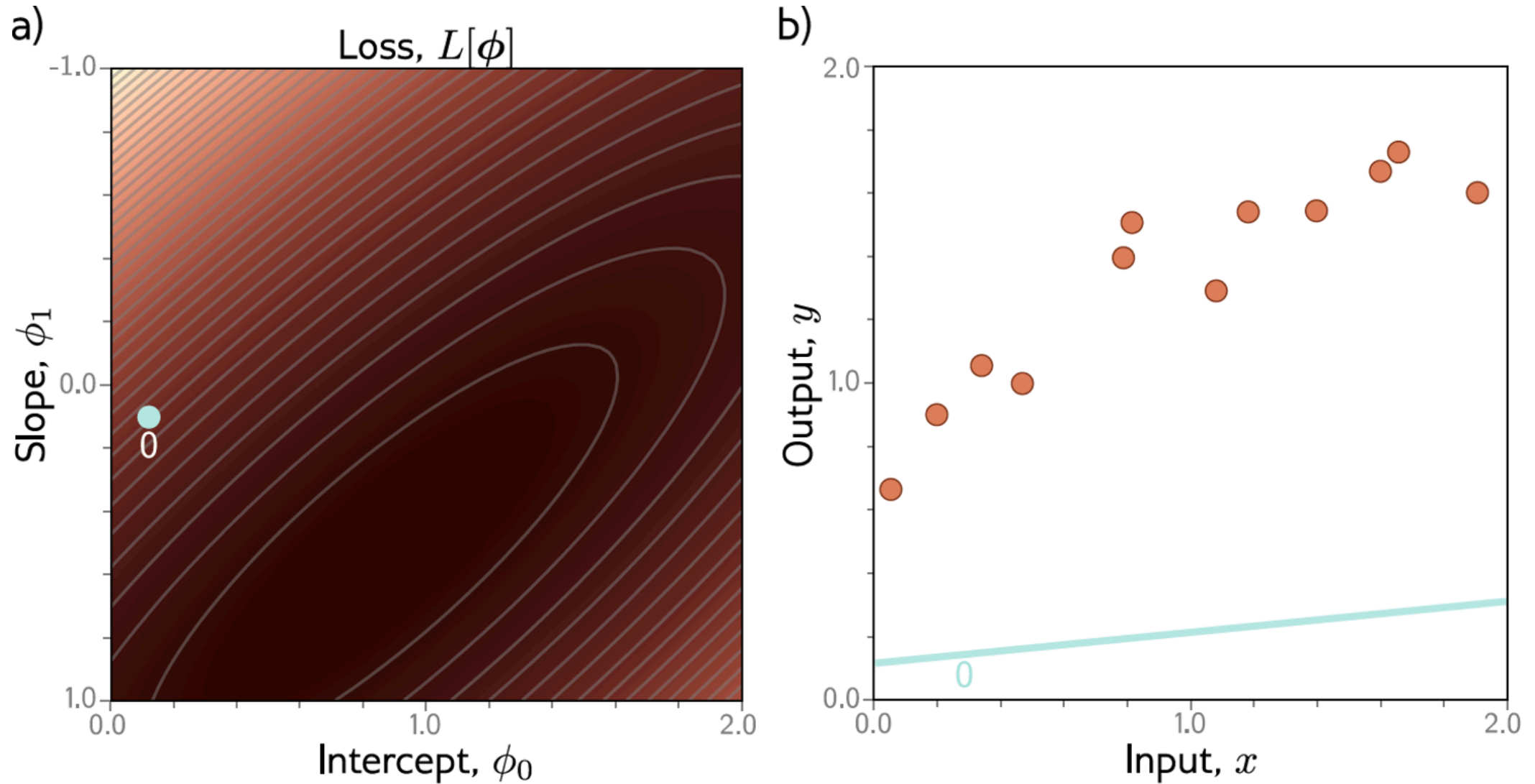
a)



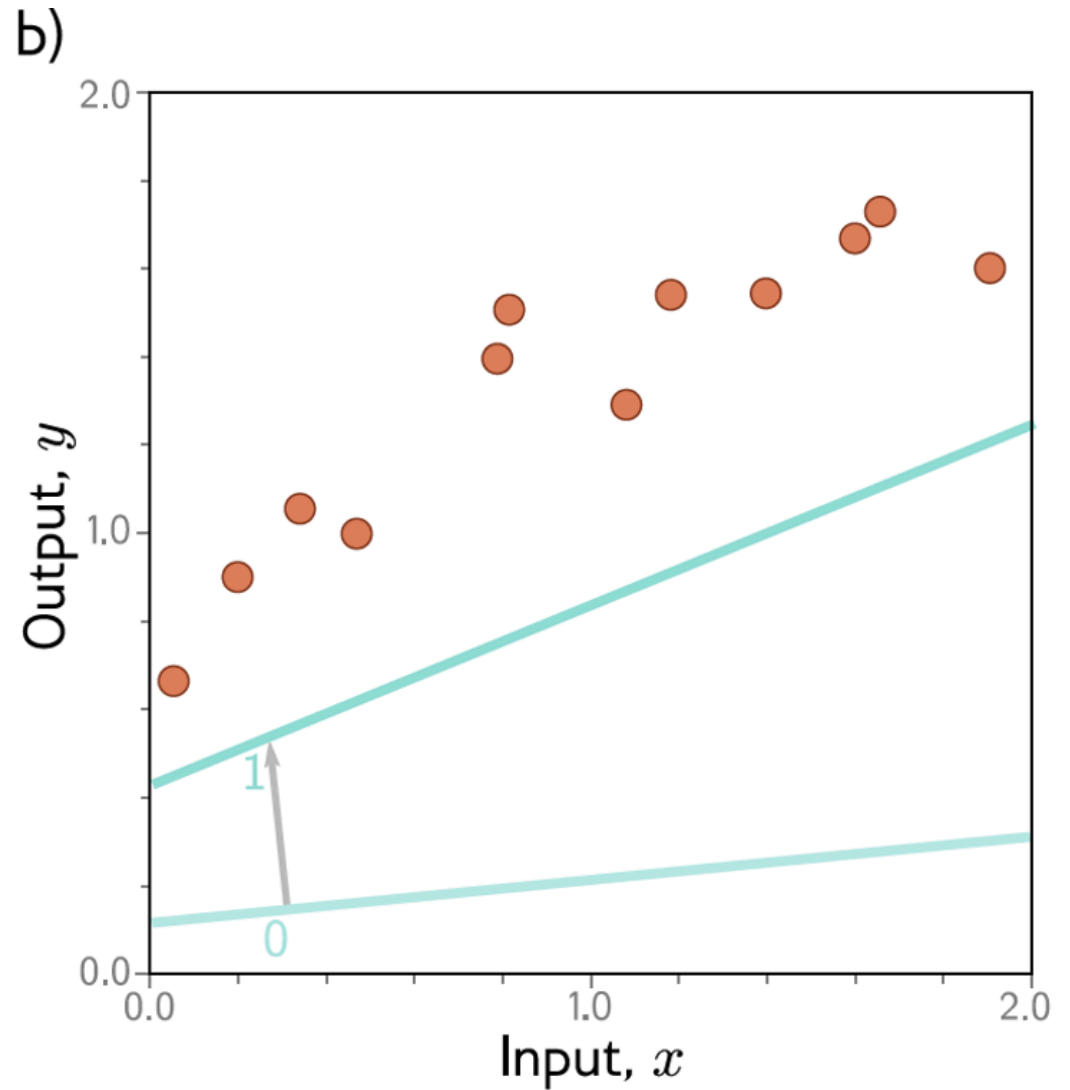
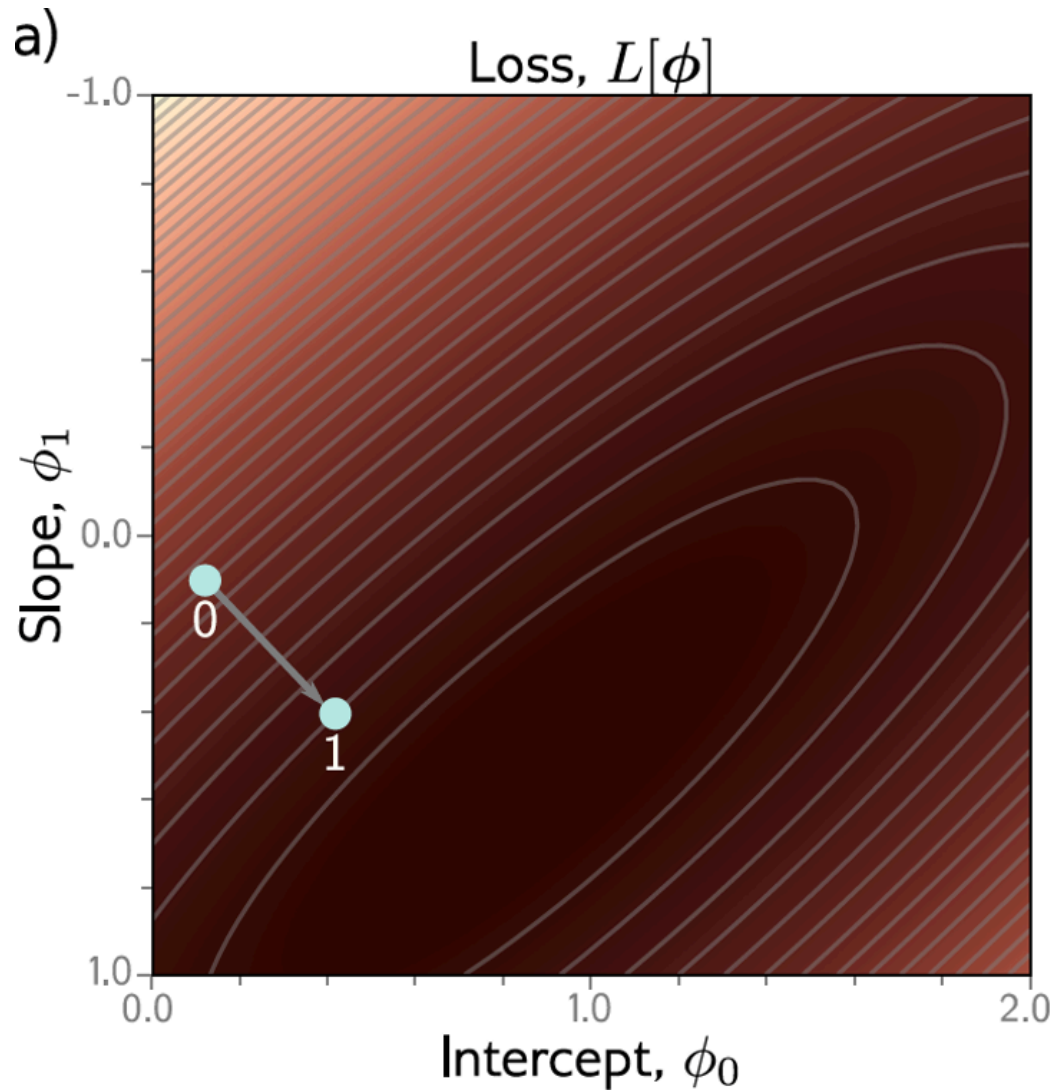
b)



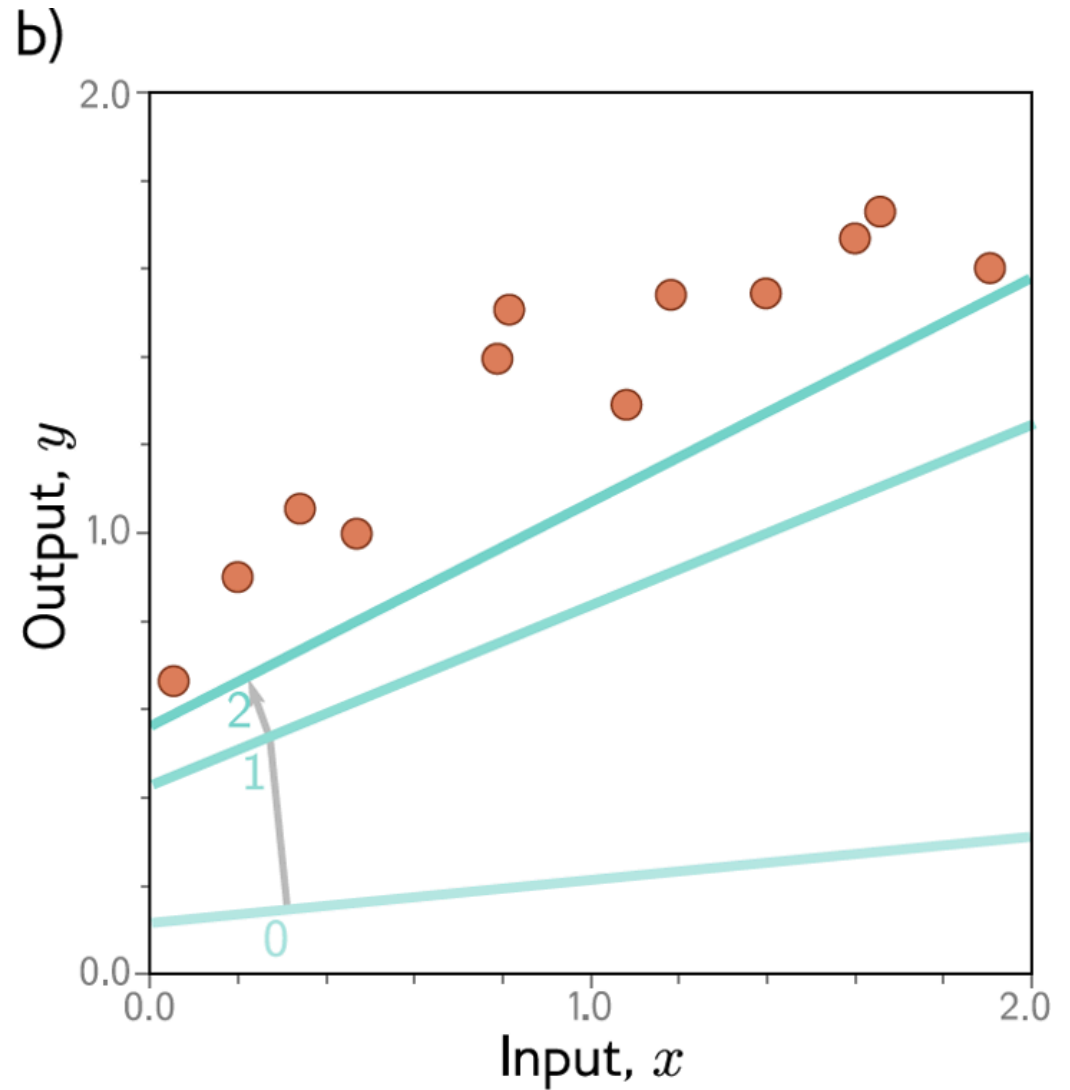
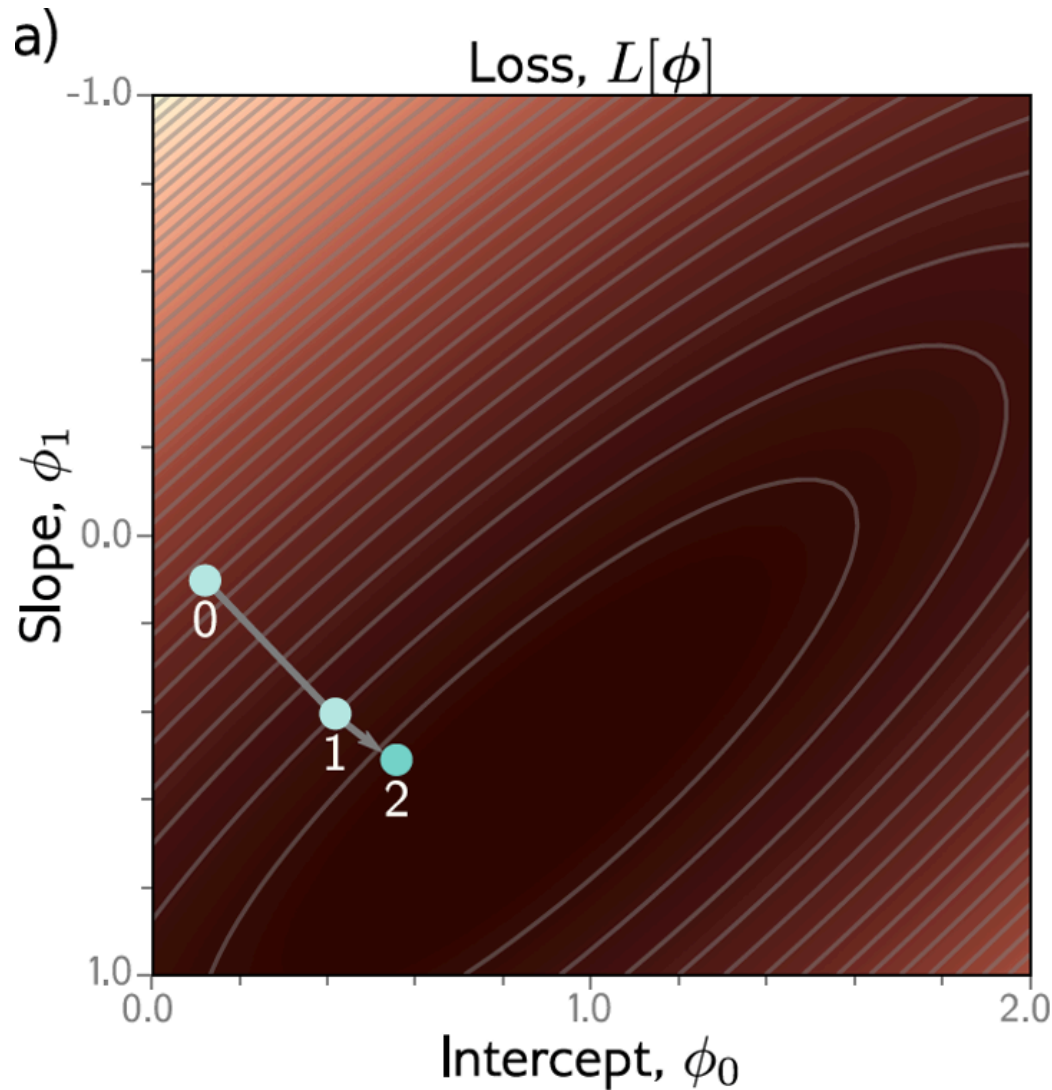
Example: 1D Linear regression model



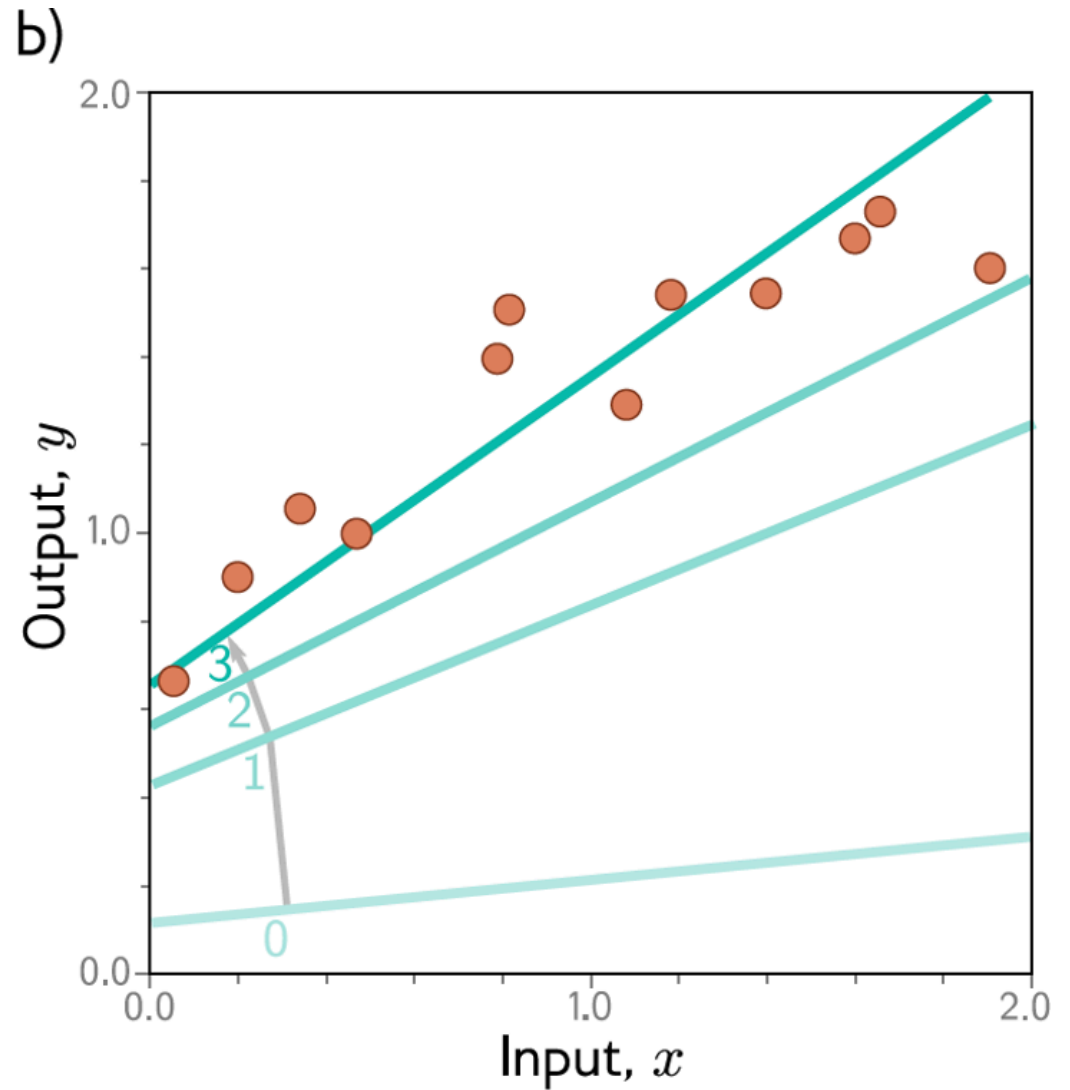
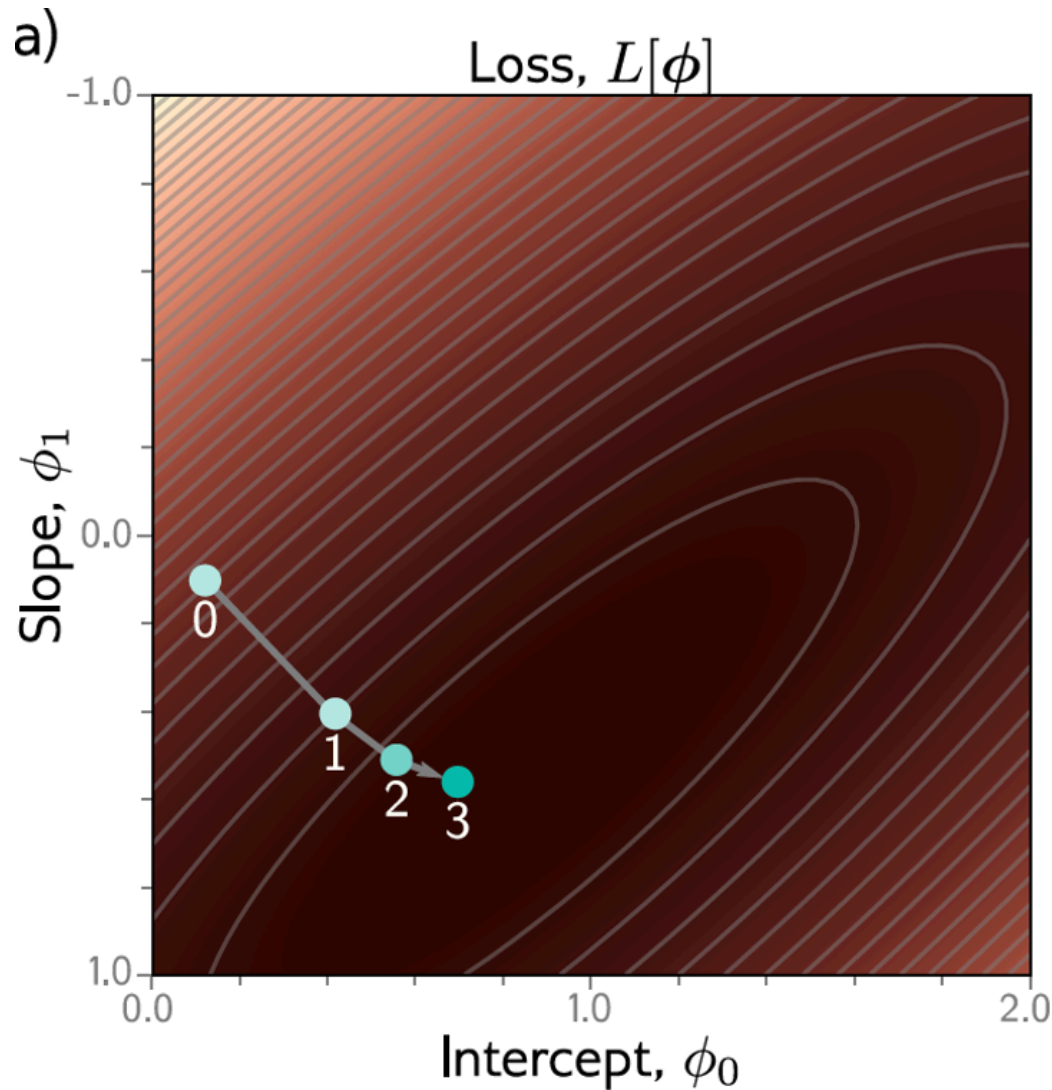
Example: 1D Linear regression model



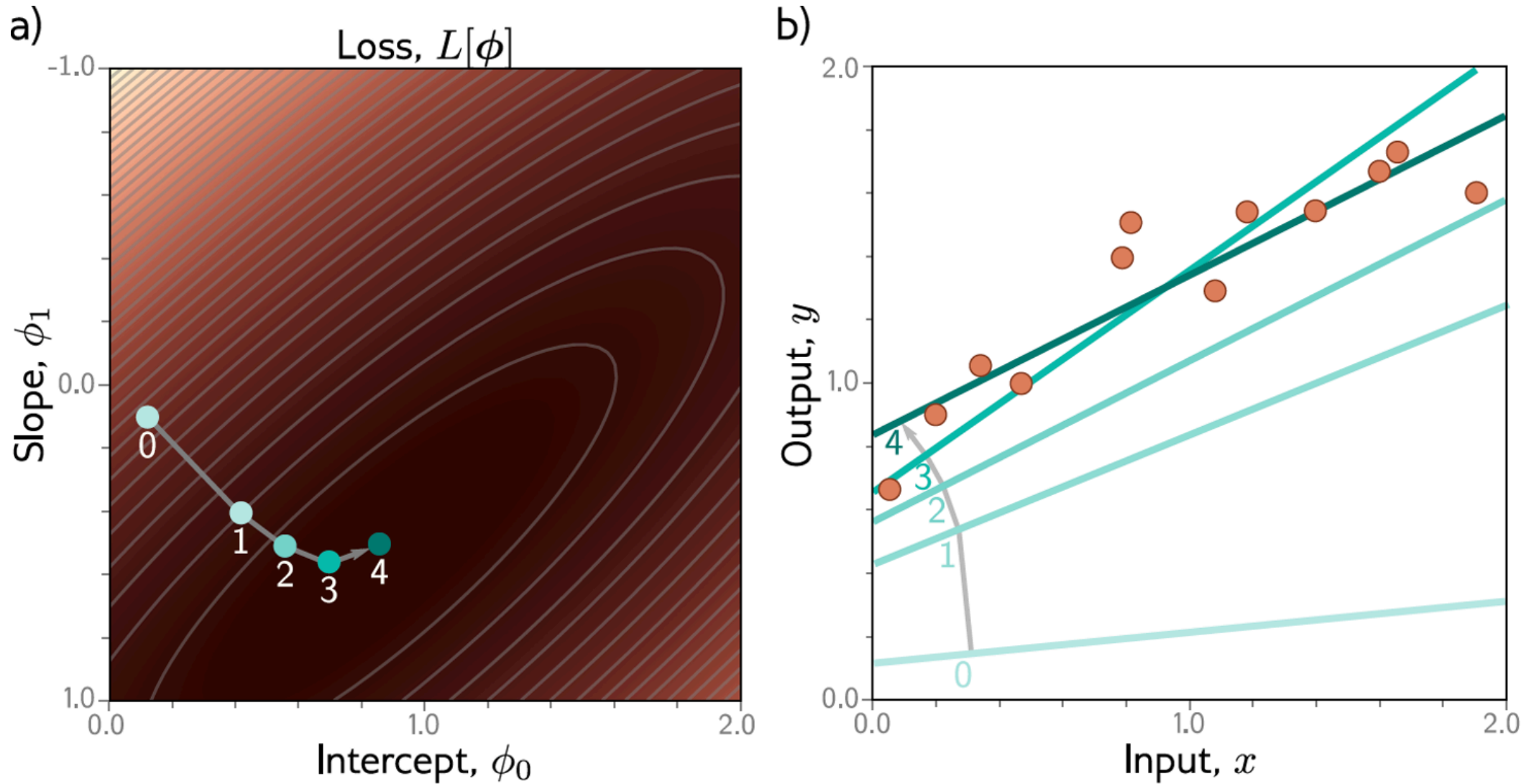
Example: 1D Linear regression model



Example: 1D Linear regression model



Example: 1D Linear regression model



Possible objections:

- But you can fit the line model in closed form!
 - Yes – but we won't be able to do this for more complex models
- But we could exhaustively try every slope and intercept combo!
 - Yes – but we won't be able to do this when there are a million parameters

Supervised learning overview

- **Supervised learning model** = mapping from one or more inputs to one or more outputs
- Model is a mathematical equation (e.g., $y = \phi_1 x + \phi_0$), or family of equations (e.g., neural networks)
- Model also includes **parameters** (e.g., ϕ_1 and ϕ_0 from above)
- Parameters affect outcome of equation
- **Training** a model = finding parameters that predict outputs “well” from inputs for a **training dataset** of input/output pairs

Supervised learning overview

- parameter estimation for non-linear models (e.g., psychometric functions)
- training the model by reducing RMSE

Example: non-linear model (Psychometric functions)

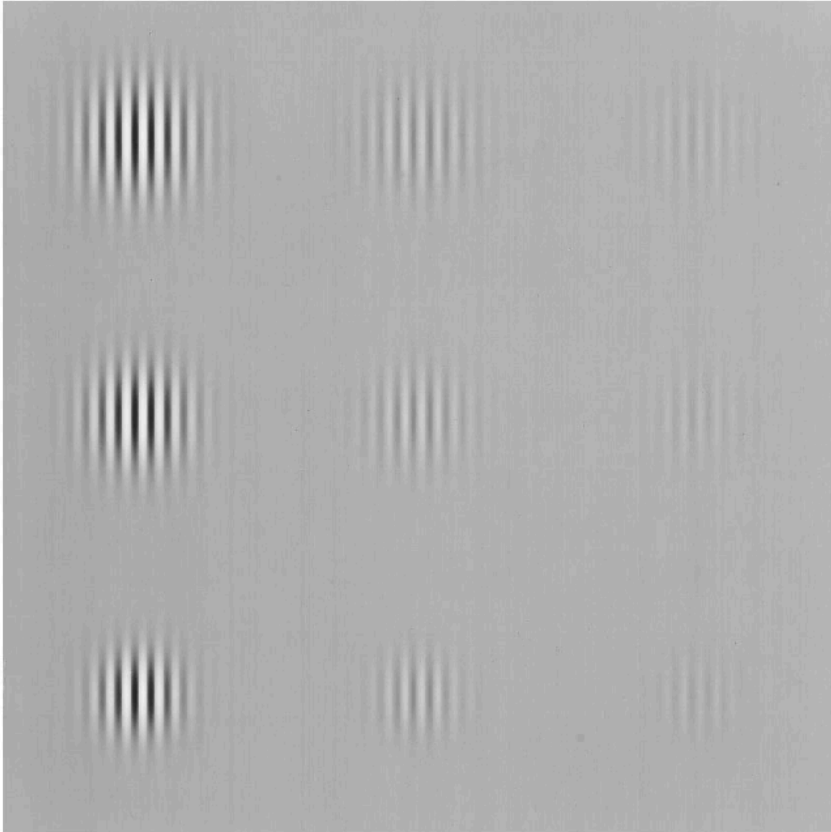
- The **psychometric function** is a summary of the relation between performance in a classification task (such as the ability to detect or discriminate between stimuli) and stimulus level/intensity (*Knoblauch & Maloney, 2012, Chapter 4*)

What are psychophysical data?

- Psychophysical data are obtained from a psychophysical experiment. Commonly a researcher wants to associate external stimulus variables with internal representations, and measures performance related to manipulations of these external variables.

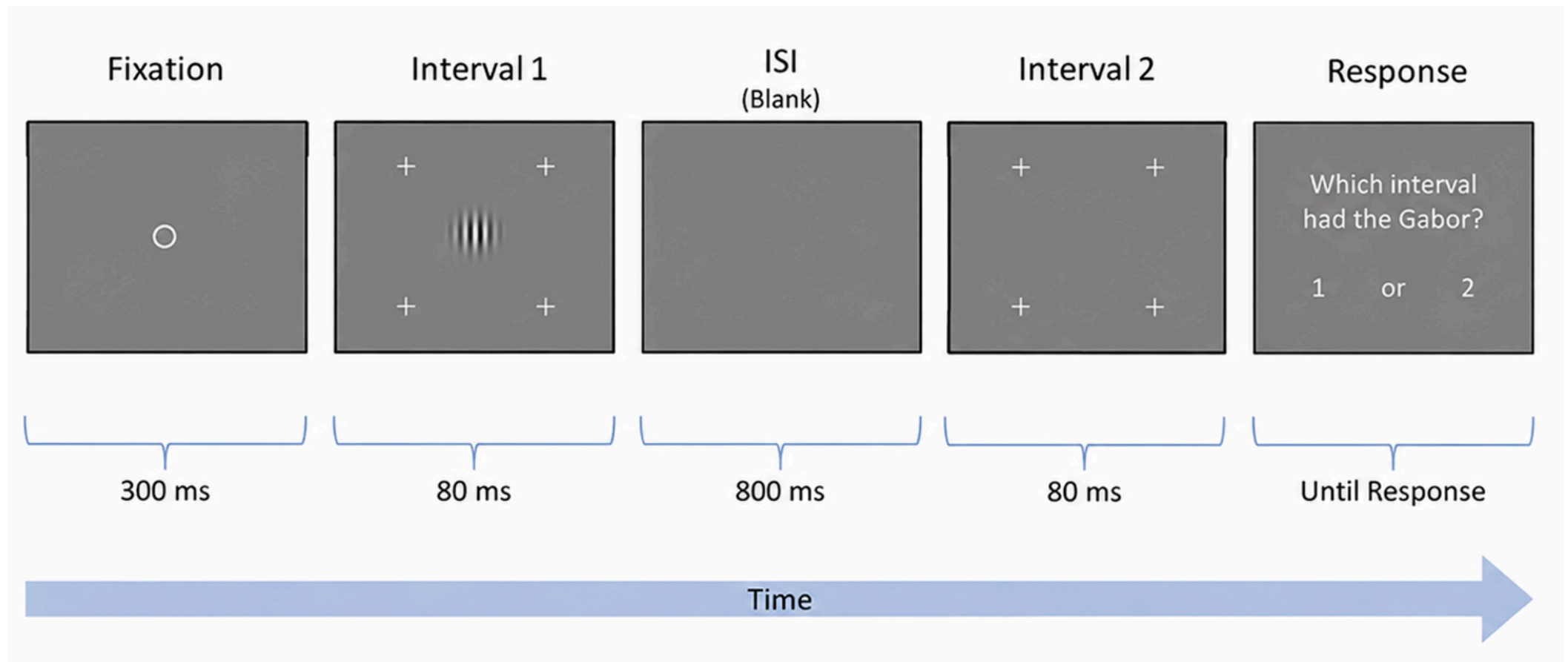
What are psychophysical data?

Contrast detection task



Decreasing contrast values (harder to detect)——>

What are psychophysical data? Example trials



Psychophysical data

- Classic psychophysics setup: on each trial, the participant says whether the stimulus was in interval 1 or 2. We repeat this many times at each of several contrast levels, and the proportion of “correct” answers is what traces out the curve

Psychophysical data

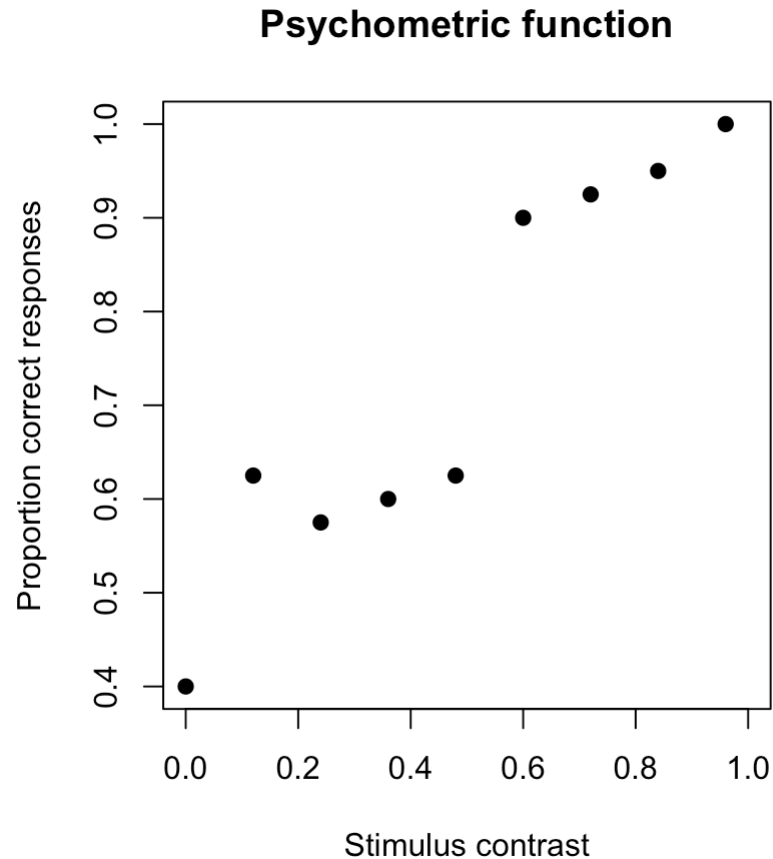
```
1 contrast <- seq(0,1,0.12)
2 n_trials <- rep(40, 9)
3 n_correct <- c(16, 25, 23, 24, 25, 36, 37, 38, 40)
4 prop_correct <- n_correct / n_trials
5
6 data <- data.frame(contrast, n_correct, n_trials, prop_correct)
7 data
```

	contrast	n_correct	n_trials	prop_correct
1	0.00	16	40	0.400
2	0.12	25	40	0.625
3	0.24	23	40	0.575
4	0.36	24	40	0.600
5	0.48	25	40	0.625
6	0.60	36	40	0.900
7	0.72	37	40	0.925
8	0.84	38	40	0.950
9	0.96	40	40	1.000

Plotting the data

```
1 old_par <- par(pty = "s")
2 plot(data$contrast, data$prop_correct,
3       pch = 19,
4       xlim = c(0, 1),
5       ylim = c(0.4, 1),
6       xlab = "Stimulus contrast",
7       ylab = "Proportion correct responses",
8       main = "Psychometric function")
9 par(old_par)
```

Plotting the data (result)



- The curve rises from near 0.5 to near 1 as intensity increases — an S-shape
- To describe this curve with a model, we need just two numbers

The model

$$p(x) = 1 - e^{-(x/\alpha)^\beta}, \quad x \geq 0$$

This is the **Weibull cumulative distribution function** — see [Wikipedia: Weibull distribution](#) for the full mathematical background.

- **alpha** (threshold)
- **beta** (slope)

The model

- the Weibull CDF falls between 0 and 1. But our data falls between 0.5 and 1. We can thus squish our psychometric model to fall in the region of 0.5 to 1 as follows:

$$p(x) = a + b * (1 - e^{-(x/\alpha)^\beta}), \quad x \geq 0$$

- **a** and **b** are also parameters

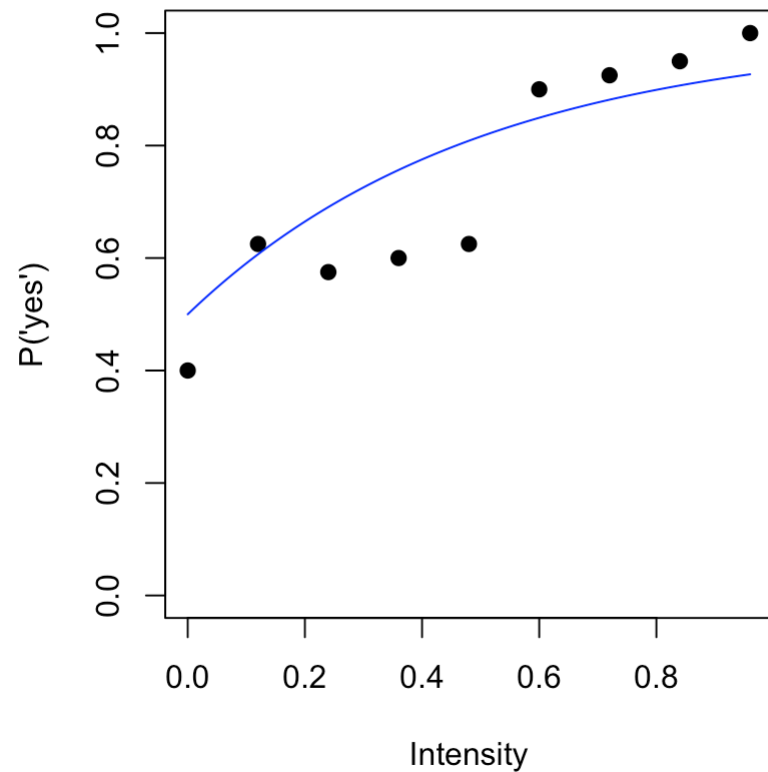
The model

- Here, **b** compresses the function to all between 0 and **b**, and **a** adds a constant such that the output of the function ranges from [**a**,**b**]
- Given our data falls between 0.5 and 1, we assign **a**, and **b** to be 0.5

```
1 psychometric_model <- function(alpha, beta, x) {  
2   y <- rep(0, length(x))  
3   valid <- x >= 0  
4   y[valid] <- 0.5 + 0.5*(1 - exp(-(x[valid] / alpha)^beta))  
5   y  
6 }
```

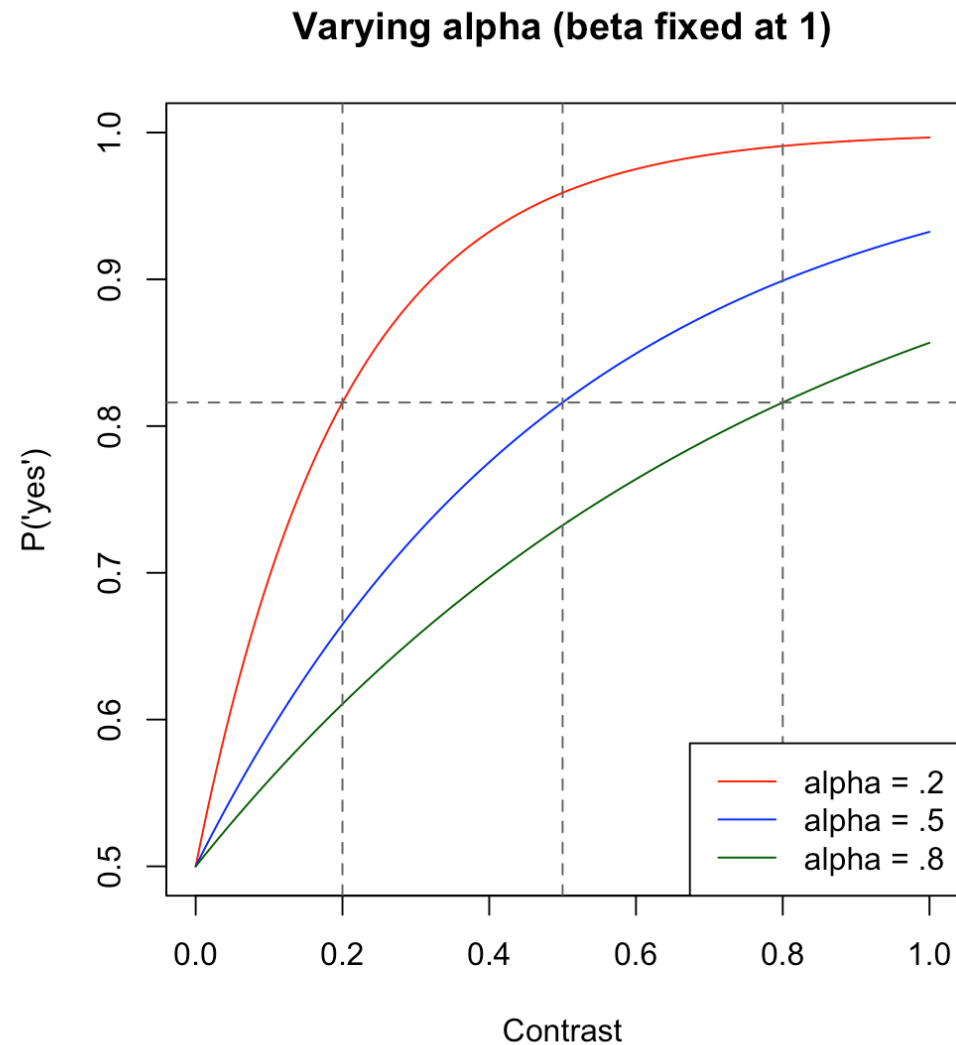
The model

Let's see whether a plausible guess looks like our data:



```
1 old_par <- par(pty = "s")
2 plot(data$contrast, data$prop_correct, pch =
3       xlab = "Intensity", ylab = "P('yes')")
4 curve(psychometric_model(.5, 1, x), add = TRUE)
5 # Restore original default layout parameters
6 par(old_par)
```

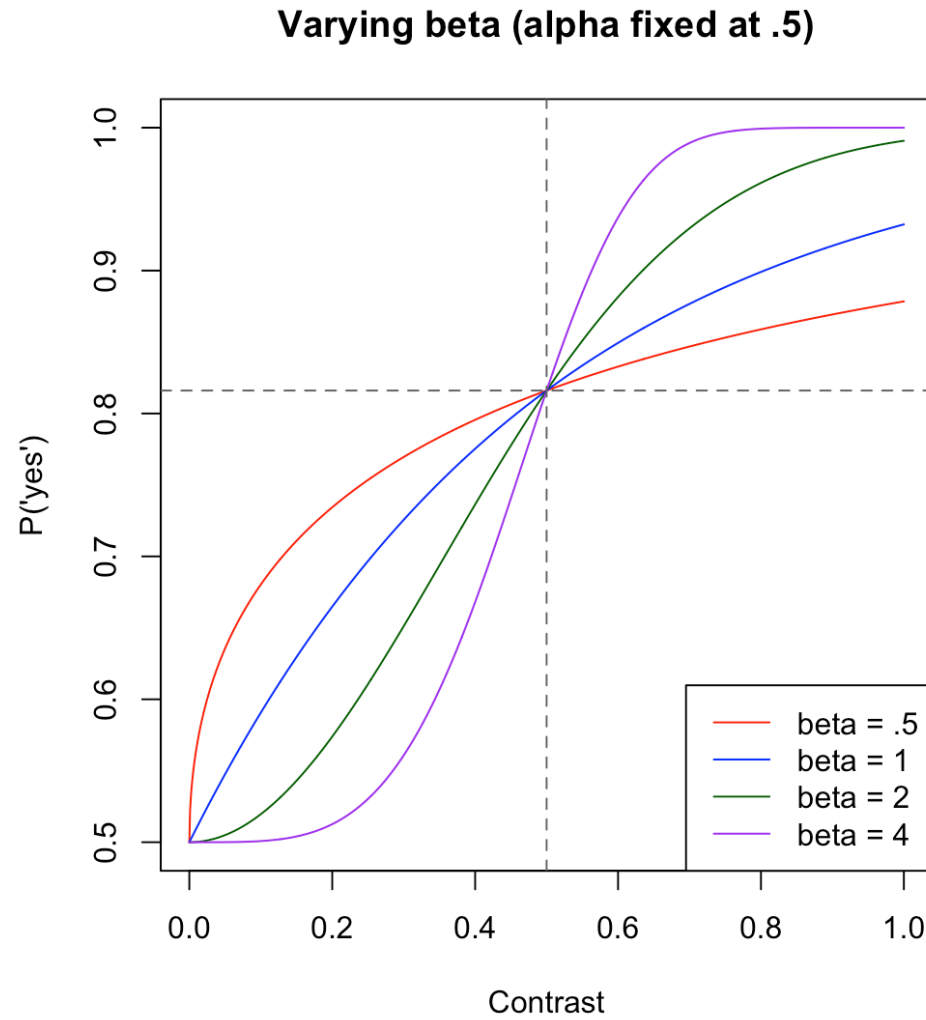
alpha: a threshold / location parameter



alpha: a threshold / location parameter

- **alpha** is a **scale** parameter, not a pure shift: increasing it moves the curve right *and* stretches its width — notice the three curves above aren't just shifted copies of each other, the $\alpha=.8$ curve is visibly more gradual than the $\alpha=.3$ curve
- A useful fixed point regardless of **beta**: the curve always reaches about **82%** at $x = \alpha$ (a property of the Weibull's mathematical form)
- Psychologically: how sensitive the person is overall — a lower **alpha** means they reach a given detection level at a fainter stimulus

beta: a slope or steepness parameter



beta: a slope or steepness parameter

- **beta** controls **how sharply** the curve rises — all three curves pass through the same 82% point (at $x = .5$, i.e., when $x = \alpha$) but climb at very different rates
- Psychologically: how *reliably* the person's responses track the physical stimulus

Why don't the dots sit exactly on the curve?

- The model gives a single, *deterministic* prediction for each intensity
- But real data always has some trial-to-trial noise
- We can think of the data as: **model prediction + random error**

$$\text{observed} = \text{model}(\alpha, \beta, x) + \text{error}$$

- This is *why* fitting can get close but never perfect

How wrong is a guess? — RMSD

Following the textbook, we use the root mean squared deviation (RMSD) rather than a raw sum of squares:

$$\text{RMSD} = \sqrt{\frac{\sum_{j=1}^J (d_j - p_j)^2}{J}}$$

```
1 rmsd <- function(params, data) {  
2   alpha <- params[1]  
3   beta <- params[2]  
4   predicted <- psychometric_model(alpha, beta, data$contrast)  
5   sqrt(mean((data$prop_correct - predicted)^2))  
6 }  
7  
8 rmsd(c(.1, 2), data)    # a bad-ish guess
```

```
[1] 0.2525092
```

```
1 rmsd(c(.5, 1), data)    # a better guess
```

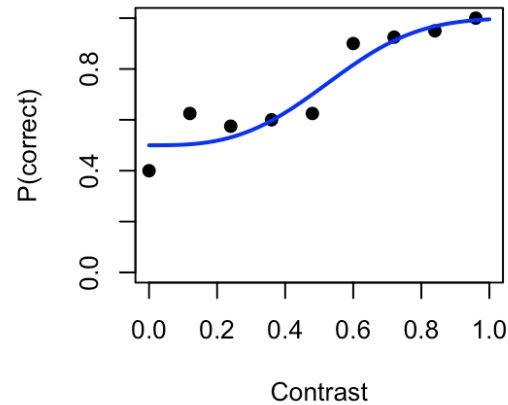
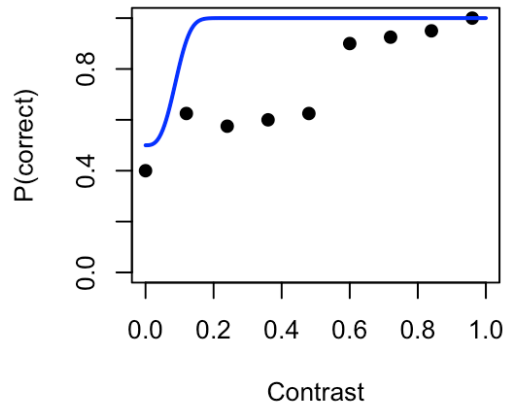
```
[1] 0.1019866
```


How wrong is a guess? — RMSD

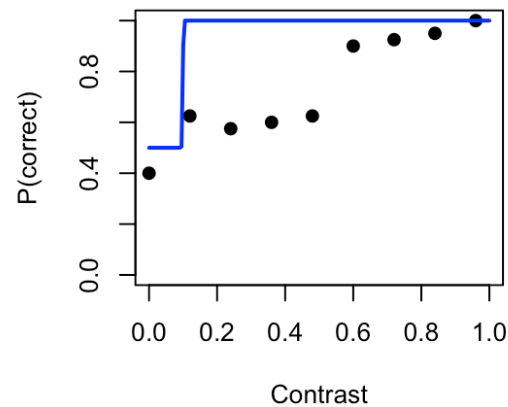
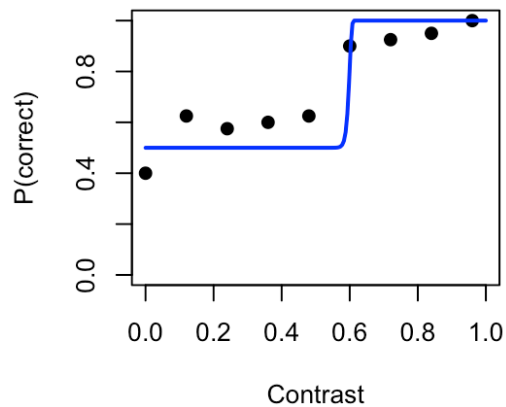
- Bigger number = worse fit; our job is to make it as small as possible
- RMSD is in the same units as the data (here, a proportion) and doesn't grow just because we collected more data points, unlike a raw sum of squares — this is why the textbook (and most of the field) prefers it

Seeing RMSD: a few deliberately bad fits

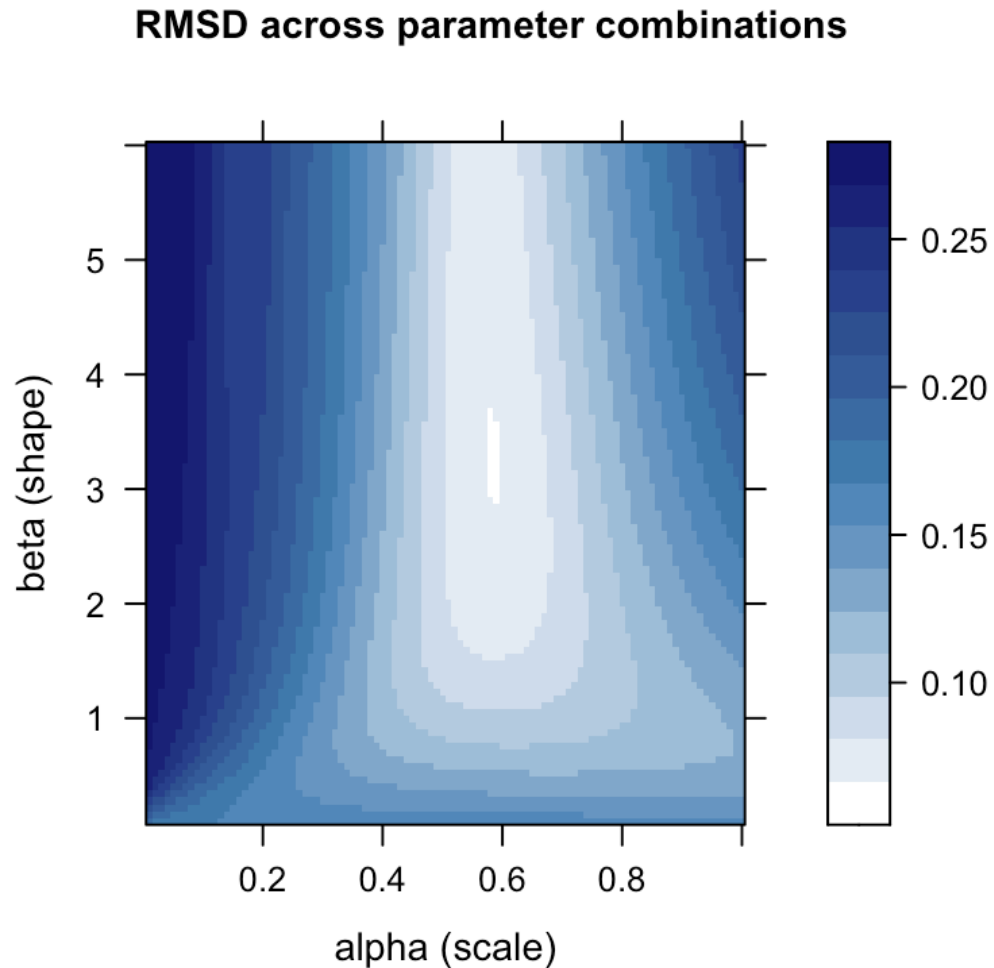
$\alpha = 0.1, \beta = 3$ (RMSD = 0.256) $\alpha = 0.6, \beta = 3$ (RMSD = 0.067)



$\alpha = 0.6, \beta = 100$ (RMSD = 0.08) $\alpha = 0.1, \beta = 100$ (RMSD = 0.26)



Seeing the cost surface



- light blue = low RMSD = good fit
- Our best-fitting combination sits at the bottom of that “valley”

Grid search: try everything

```
[1] 10000      3
```

	alpha	beta	rmsd
1	0.01	0.1	0.1874534
2	0.02	0.1	0.1813431
3	0.03	0.1	0.1779529
4	0.04	0.1	0.1756476
5	0.05	0.1	0.1739232
6	0.06	0.1	0.1725588
7	0.07	0.1	0.1714381
8	0.08	0.1	0.1704929
9	0.09	0.1	0.1696795
10	0.10	0.1	0.1689684

```
1 best <- grid[which.min(grid$rmsd), ]  
2 best
```

	alpha	beta	rmsd
5259	0.59	3.19899	0.06611321

- we're testing $100 \times 100 = 10,000$ combinations of 'alpha' and 'beta' in this case with its RMSD already computed

Grid search doesn't scale

# parameters	grid points per parameter	total combinations to test
2 (today's model)	100	10,000
4	100	100,000,000
10 (a small neural network)	100	10^{20}

- Two parameters: totally fine on a laptop
- A handful more: computationally impossible to brute-force
- We need something smarter that *searches intelligently* instead of trying everything

What is `optim()` actually doing?

- Imagine the error surface carved out of wood, and you drop a marble onto it. The marble rolls downhill and comes to rest at the bottom



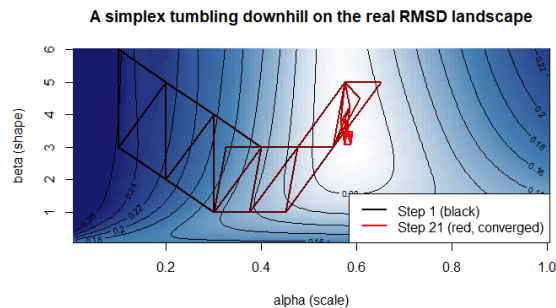
What is `optim()` actually doing?

- Start somewhere (your initial guess), move downhill, repeat until no nearby step improves things
- Much faster than grid search — it doesn't need to check every point on the map
- But: it can walk straight into a small valley that isn't the lowest one overall — a **local minimum**
- When you call `optim()`, its default method is the **Nelder-Mead Simplex** algorithm (Nelder & Mead, 1965).



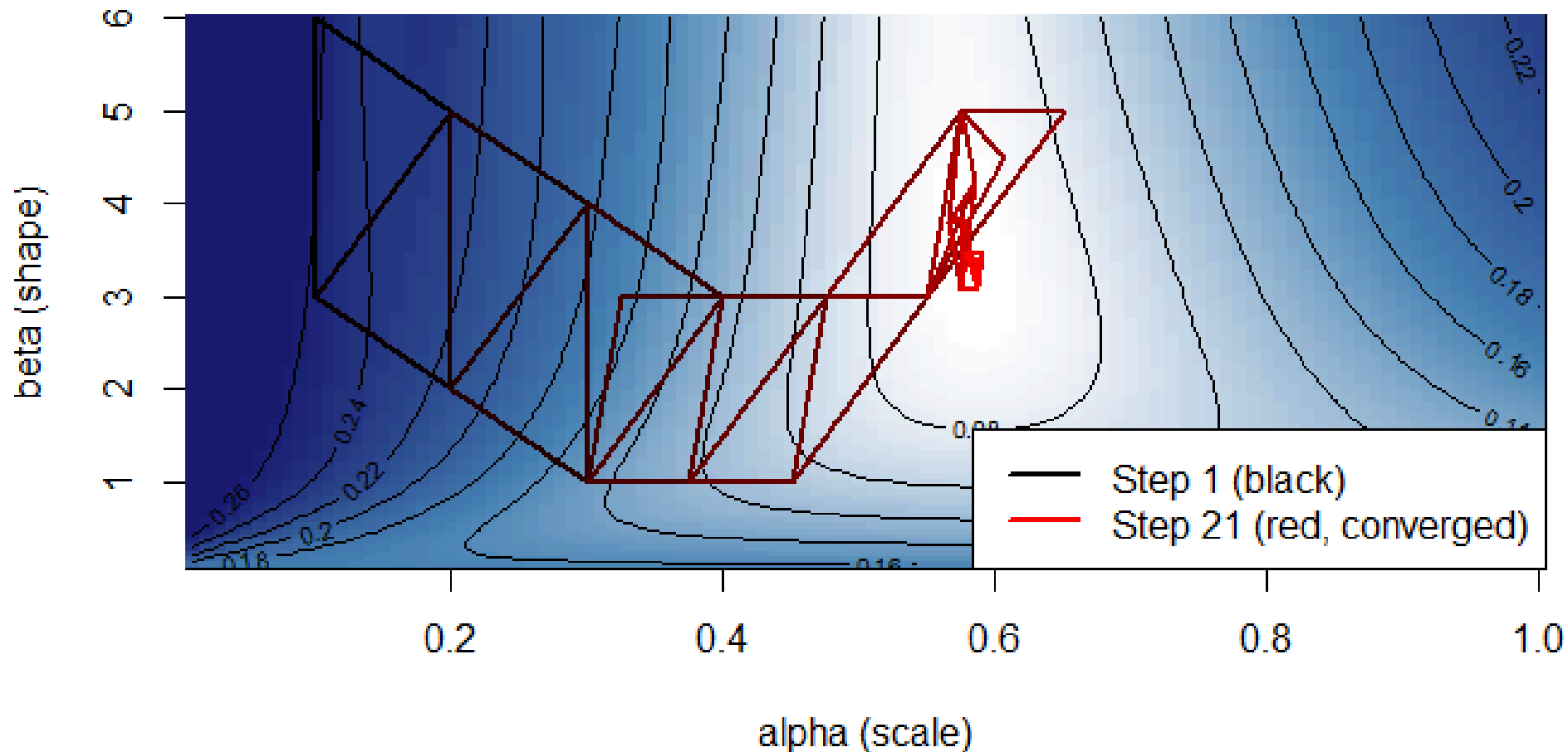
Meet the method: Nelder-Mead Simplex

- A “simplex” is a simple geometric shape: for 2 parameters, it’s a **triangle**, with each corner being one candidate (**alpha**, **beta**) combination to test.
- At each step, the corner with the *worst* fit is moved: either **reflected** to the opposite side of the triangle (and possibly stretched further, if that direction looks promising), or the whole triangle **contracts** inward if reflection doesn’t help
- Repeating this, the triangle “tumbles” downhill and eventually shrinks to a single point, the best-fitting estimate



Meet the method: Nelder-Mead Simplex

A simplex tumbling downhill on the real **RMSD** landscape



optim(): automatic search

```
1 fit_rmsd <- optim(  
2   par = c(alpha = 4, beta = 1),  
3   fn = rmsd,  
4   data = data,  
5   method = "Nelder-Mead"  
6 )  
7 fit_rmsd$par
```

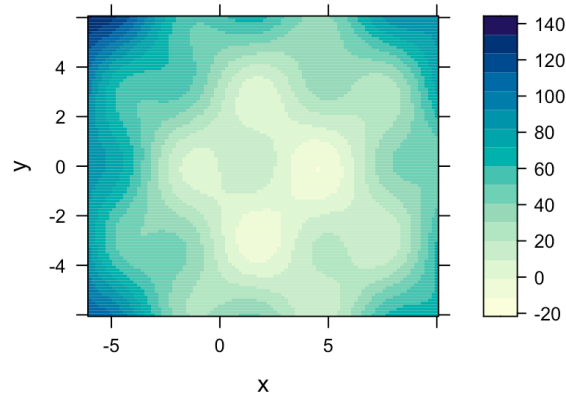
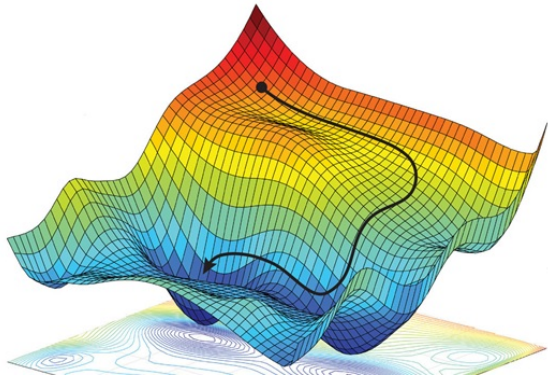
```
      alpha      beta  
0.5851889 3.2469308
```

```
1 fit_rmsd$value
```

```
[1] 0.06606295
```

Limitations: getting stuck in a local minimum

A landscape with multiple valleys



Not every landscape looks like our psychometric-function example. Here are examples with a bumpier landscapes.

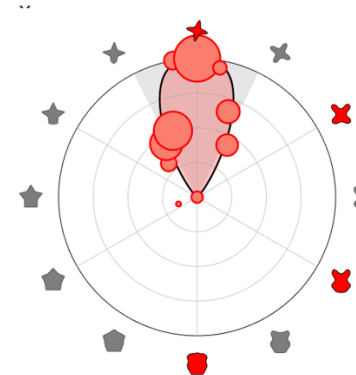
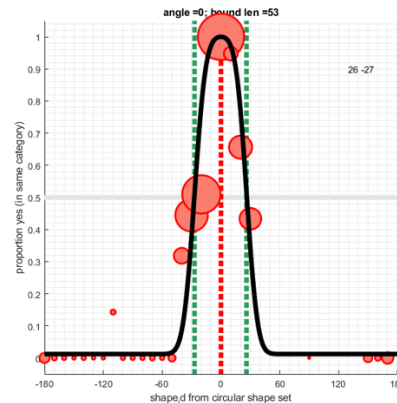
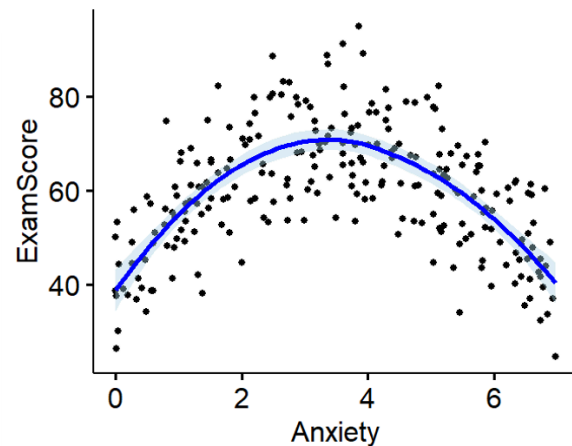
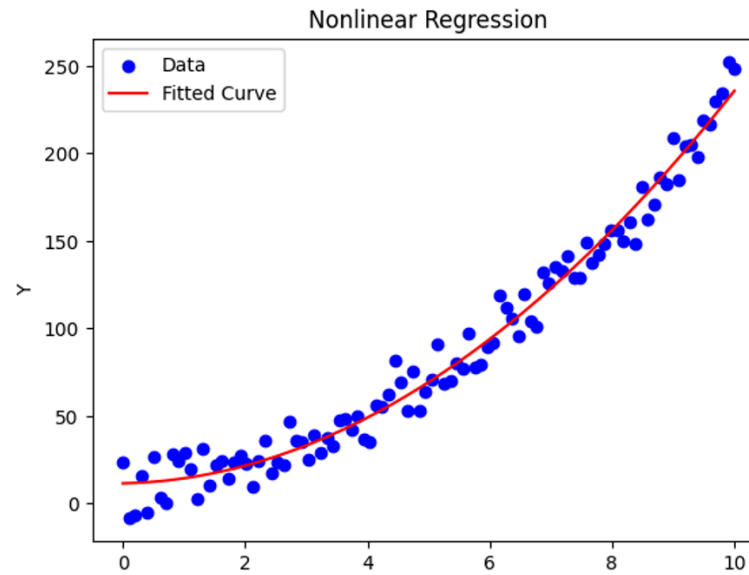
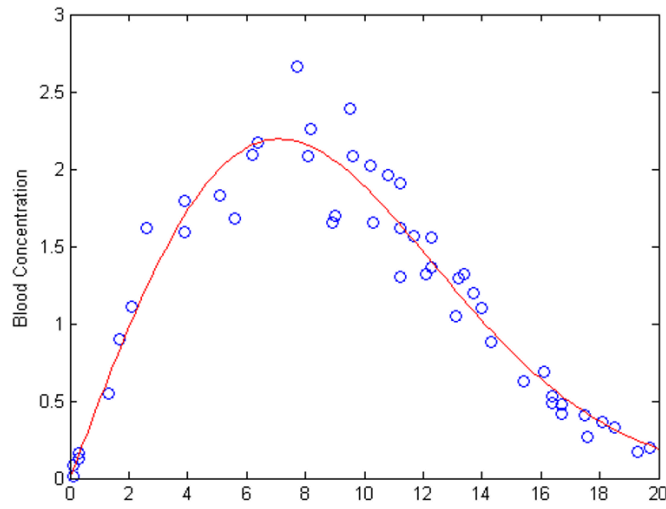
Limitations: getting stuck in a local minimum

- Several dark patches = several **local minima**; the single darkest patch overall is the **global minimum** — what we actually want
- There's no guarantee that what `optim()` finds is the global minimum rather than a local one — this is true of essentially all parameter estimation methods, not just Simplex
- One practical mitigation: run `optim()` several times from different starting values; if they all agree, that's good evidence you've found the global minimum

Limitations: struggling with many parameters

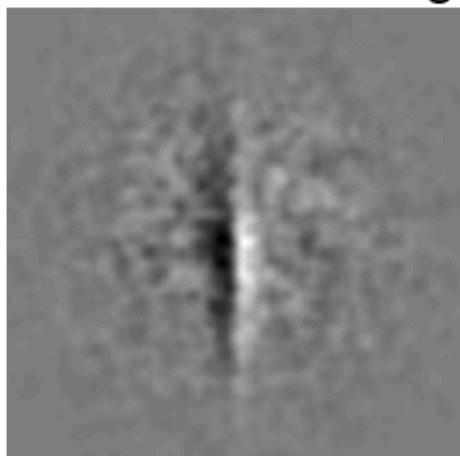
- Simplex becomes inefficient and unreliable with roughly >5 parameters
- We'll meet methods much better suited to many-parameter models in the next class

Today: Estimating model parameters from data. Example

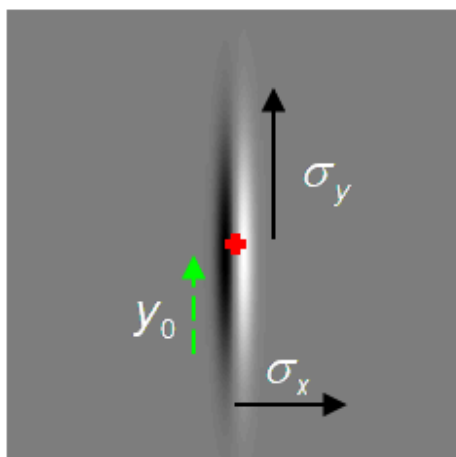


Today: Estimating model parameters from data. Example

Human Data:
Classification Image

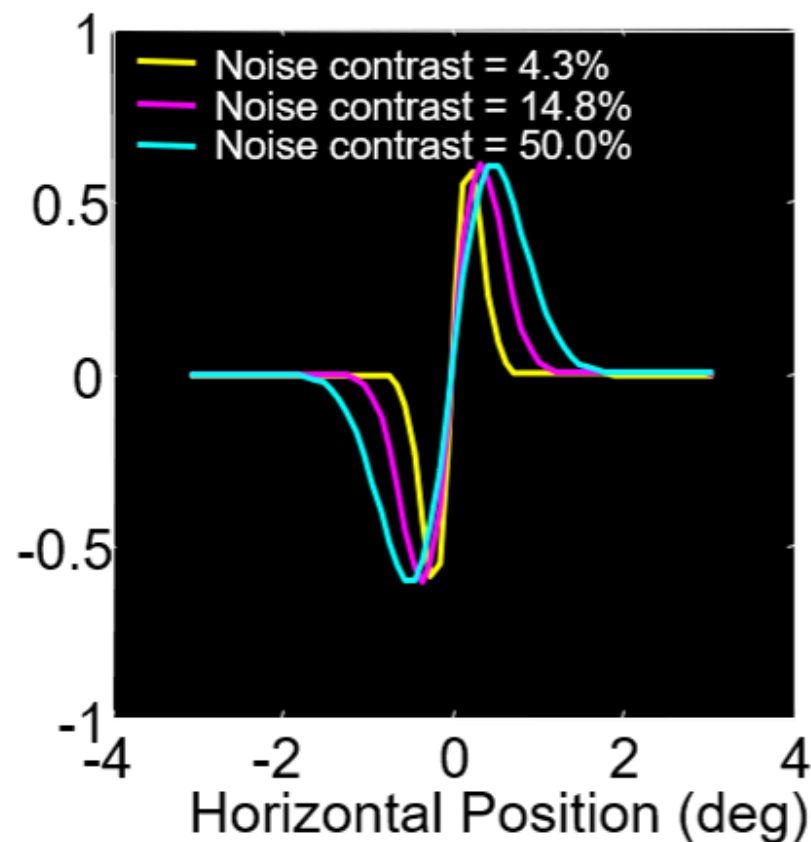


Model



Receptive field model:

$$f(x, y) = x \exp(-x^2 / 2\sigma_x^2 - (y - y_0)^2 / 2\sigma_y^2)$$



Summary

- A model has parameters, and each parameter can be tweaked to best fit the data
- $\text{Data} = \text{model} + \text{error}$ — that's why a fit is never perfect
- RMSD $\rightarrow$ grid search $\rightarrow$ `optim()`'s default Nelder-Mead simplex, which tumbles downhill but can get stuck in a local minimum on harder landscapes, and doesn't scale well beyond about 5 parameters. Both limitations we'll revisit when methods better suited to complex, many-parameter landscapes come up in the neural network sessions